Unit III

3 Classification

Syllabus

Introduction : *Need of Classification, Types of Classification (Binary and Multiclass), Binary-vs-Multiclass Classification, Balanced and Imbalanced Classification Problems.*

Binary Classification : *Linear Classification model, Performance Evaluation - Confusion Matrix, Accuracy, Precision, Recall, F measures.*

Multiclass Classification : *One-vs-One and One-vs-All classification techniques, Performance Evaluation- Confusion Matrix, Per Class Precision, Per Class Recall.*

Classification Algorithms : *K Nearest Neighbor, Linear Support Vector Machines (SVM) - Introduction, Soft Margin SVM, Kernel functions - Radial Basis Kernel, Gaussian, Polynomial, Sigmoid.*

Contents

3.1 Introduction of Classification

3.2 Types of Classification : Binary Classification

3.3 Performance Evaluation

3.4 Multiclass Classification

3.5 Performance Evaluation : Micro-average Method

3.6 Classification Algorithms : K Nearest Neighbor

3.7 Linear Support Vector Machines (SVM)

3.8 Kernel Functions

3.1 Introduction of Classification

- Classification predicts categorical labels (classes), prediction models continuous-valued functions. Classification is considered to be supervised learning.
- Classifies data based on the training set and the values in a classifying attribute and uses it in classifying new data. **Prediction** means models continuous-valued functions, i.e., predicts unknown or missing values.
- Preprocessing of the data in preparation for classification and prediction can involve data cleaning to reduce noise or handle missing values, relevance analysis to remove irrelevant or redundant attributes, and data transformation, such as generalizing the data to higher level concepts or normalizing data.
- Fig. 3.1.1 shows the classification.

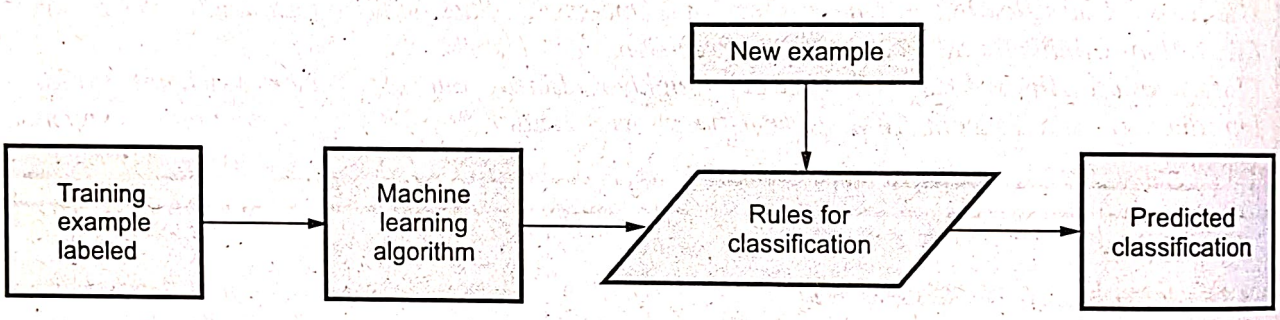


Fig. 3.1.1 Classification

Aim : To predict categorical class labels for new samples.

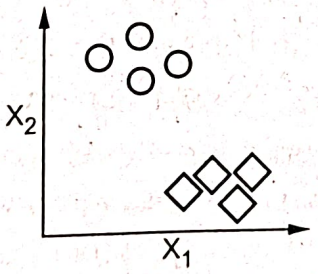
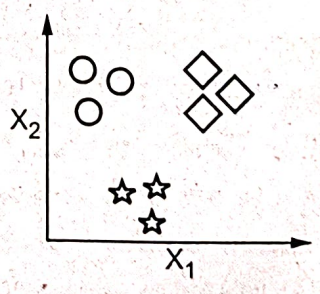
Input : Training set of samples, each with a class label.

Output : Classifier is based on the training set and the class labels.

- **Prediction** is similar to classification. It constructs a model and uses the model to predict unknown or missing value.
- Classification is the process of finding a model that describes and distinguishes data classes or concepts, for the purpose of being able to use the model to predict the class of objects whose class label is unknown. The derived model is based on the analysis of a set of training data.
- Classification and prediction may need to be preceded by relevance analysis, which attempts to identify attributes that do not contribute to the classification or prediction process.

- Numeric prediction is the task of predicting continuous values for given input. For example, we may wish to predict the salary of college employee with 15 years of work experience, or the potential sales of a new product given its price.
- Some of the classification methods like back-propagation, support vector machines, and k-nearest-neighbor classifiers can be used for prediction.

3.1.1 Binary - vs - Multiclass Classification

Sr. No.	Binary classification	Multiclass classification
1.	Task of classifying an item of a given set into two groups.	Task of classifying an item of a given set into multiple groups.
2.	It is common to model a binary classification task with a model that predicts a **Bernoulli probability distribution** for each example.	It is common to model a multi-class classification task with a model that predicts a **Multinoulli probability distribution** for each example.
3.	Popular algorithms that can be used for binary classification include : Logistic regression, k-nearest neighbors, decision trees, support vector machine and Naive bayes.	Popular algorithms that can be used for multi-class classification include : k-nearest neighbors, decision trees, Naive bayes, random forest and gradient boosting.
4.	**Example :** Decide if an email is spam or not.	**Example :** Decide if an email is advertisement, newsletter, phishing or personal.
5.		

3.1.2 Balanced and Imbalanced Classification Problems

Balanced classification problems

- A dataset is balanced if one set of data is equal to the second set of data.

- Balancing a dataset makes training a model easier because it helps prevent the model from becoming biased towards one class.
- Fig. 3.1.2 shows balanced class and imbalanced class.

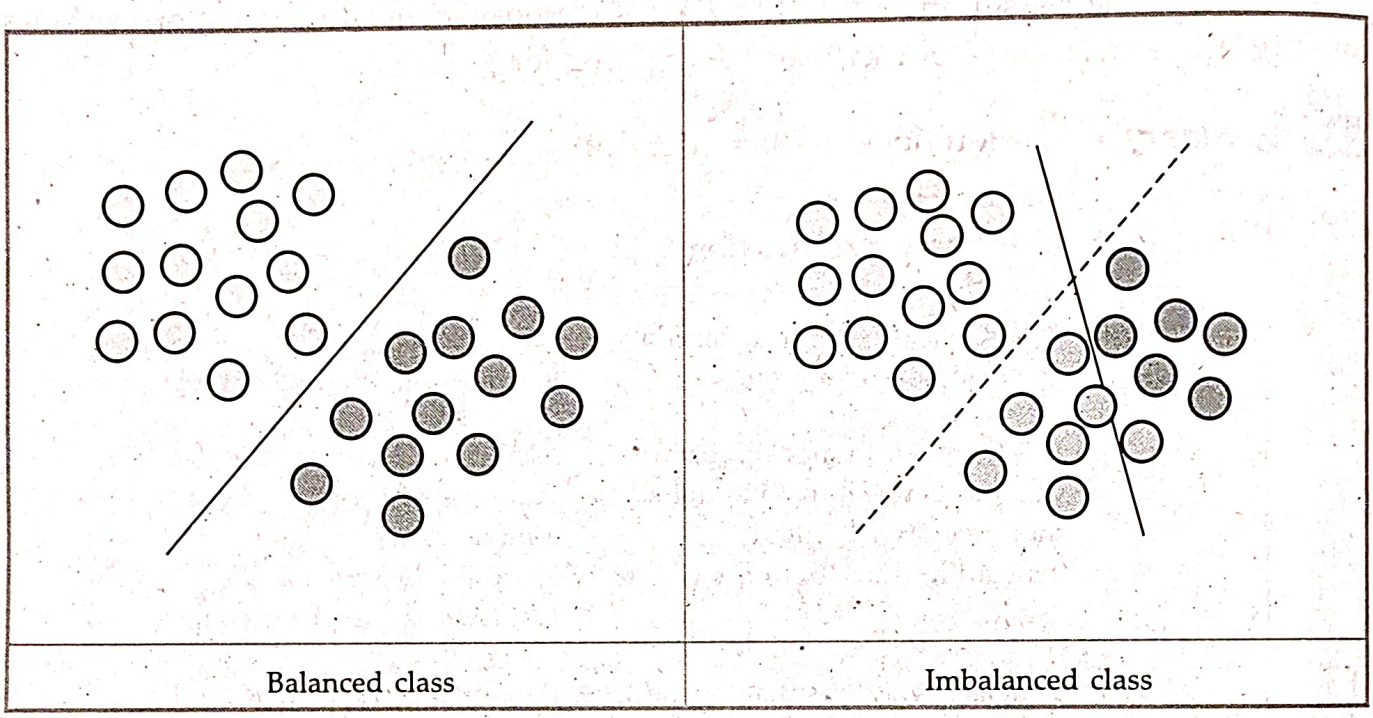


Fig. 3.1.2

Imbalanced classification problems

- A dataset is imbalanced if one set of data has high dominance over the second set of data.
- Imbalanced datasets create challenges for predictive modeling
- While learning, several techniques such as support vector machine faces great challenge in presence of imbalanced data.
- Imbalance is usually represented as ratio between the total number of occurrence in majority class and the total number of occurrence in the minority class.
- Imbalanced data typically refers to a problem with classification problems where the classes are not represented equally. For example, we may have a 3-class classification problem of set of fruits to classify as oranges, apples or pears with total 100 instances.
- A total of 80 instances are labeled with Class - 1 (Oranges), 10 instances with Class - 2 (Apples) and the remaining 10 instances are labeled with Class - 3 (Pears). This is an imbalanced dataset and the ratio of 8 : 1 : 1.
- Most classification data sets do not have exactly equal number of instances in each class, but a small difference often does not matter.

3.2 Types of Classification : Binary Classification

- If the output is a binary value; (yes/no), (+/–), (0/1), then the learning problem is referred to as a binary classification problem. Example : Learning to distinguish sport cars.
- If there are a finite number of categories, the problem is known as a **multi-class classification.**
- For a collection of objects, user task is to classify the objects into two groups based on some feature(s). For example, let us say given some pens and pencils of different types and makes, we can easily separate them into two classes, namely pens and pencils.
- This seemingly trivial task, if we take a moment would seem to involve a lot of underlying computation. The question to ask would be what is our basis for classification ? And how perhaps might we incorporate the principle to allow for efficient classification by 'intelligent algorithms'.
- Binary classification : Trying to predict a simple yes/no response. For instance, predict whether Rakshita will enjoy a course or not. Or predict whether a user review of the newest Apple product is positive or negative about the product.
- Binary classification would generally fall into the domain of supervised learning since the training dataset is labeled. Discriminant function is used to represent a classifier.
- A well known example of a binary classification problem is predicting whether an email is spam or not spam.
- In binary classification, the objective is to design a rule that assigns objects to one of 2 classes, often referred to as positive and negative, on the basis of descriptive vectors of objects.
- Formally, we will say an instance x is positive if it has associated label y =1 and negative if its label y = 0. We then define the 0 - 1 misclassification loss for a binary prediction p when the label is y to be

$$l_{01}(y, p) = \begin{cases} 1 & y \neq p \\ 0 & \text{otherwise} \end{cases}$$

- **Example of classification :** The ABC bank wants to classify its customers based on whether they are expected to pay back their approved home loans. For this purpose, the history of past customers is used to train the classifier. The classifier provides rules, which identify potentially reliable future customers. The classification rule is as follows :

a. If age = "31...40"and income = Low then credit-rating = Bad

b. If age = "31...40"and income = High then credit-rating = Very good

Classification is a two step process : **Model construction and model usage.**

1. Model construction :

- Model construction is used for describing a set of predetermined classes. Each sample is assumed to belong to a predefined class, as determined by the class label attribute.
- Training set is a set of samples. The model is represented as classification rules, decision trees, or mathematical formulae.

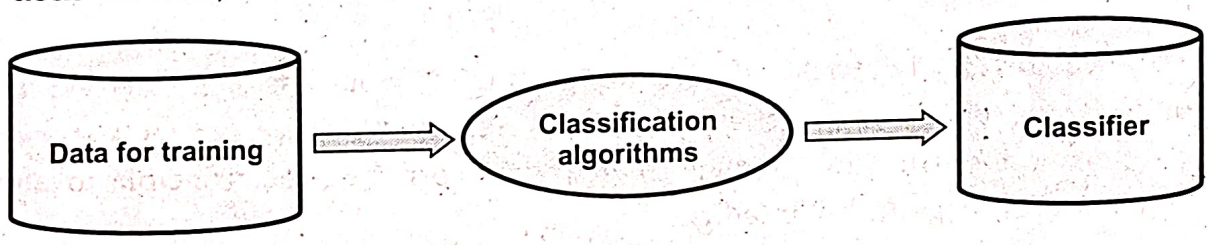


Fig. 3.2.1 Model construction

2. Model usage :

- It is used for classifying future or unknown objects. It estimates an accuracy of the constructed model. The known label of test samples is compared with the classified result from the model.
- Accuracy rate is the percentage of test set samples that are correctly classified by the model. Test set is independent of training set, otherwise over-fitting will occur.

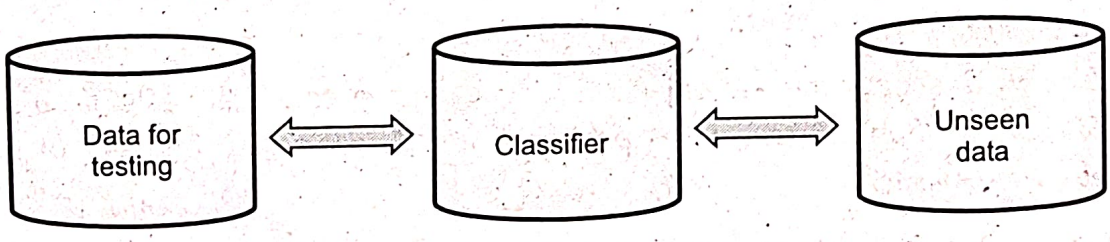


Fig. 3.2.2 Model usage

- Some examples of binary classification problems :

1. You want to train a medical diagnosis system to predict whether a patient has a given disease. You have a training set consisting of a set of patients, a set of features for those individuals, and a label saying whether or not the patient had the disease.
2. You are running an online payment service and want to determine whether or not a given transaction is fraudulent. You have a labeled training dataset of fraudulent and non-fraudulent transactions; features might include the type of transaction, the amount of money, or the time of day.

3.2.1 Linear Classification Model

- A classification algorithm (Classifier) that makes its classification based on a linear predictor function combining a set of weights with the feature vector.
- A linear classifier does classification decision based on the value of a linear combination of the characteristics. Imagine that the linear classifier will merge into it's weights all the characteristics that define a particular class.
- Linear classifiers can represent a lot of things, but they can't represent everything. The classic example of what they can't represent is the XOR function.

3.2.2 Implementation and Optimizations

- scikit-learn implements the "LogisticRegression" class, which can solve problem using optimized algorithms.
- For example, the *sklearn.linear_model.LinearRegression* estimator allows the user to specify whether or not to fit an intercept term. This is done by setting the corresponding constructor arguments of the estimator object.
- In order to immediately test the accuracy of our classification, it's useful to split the dataset into training and test sets : ***from sklearn.model_selection import train_test_split***
- Now we can train the model using the default parameters :
  from sklearn.linear_model import LogisticRegression
- It's also possible to check the quality through a cross-validation : from sklearn.model_selection import cross_val_score.
- Optimization in machine learning has a slight difference. Generally, while optimizing, we know exactly how our data looks like and what areas we want to improve. But in machine learning we have no clue how our "new data" looks like, let alone try to optimize on it.
- So in machine learning, we perform optimization on the training data and check its performance on a new validation data.

3.3 Performance Evaluation

- A binary classification rule is a method that assigns a class to an object, on the basis of its description.
- The performance of a binary classifier can be assessed by tabulating its predictions on a test set with known labels in contingency table or confusion matrix, with actual classes in rows and predicted classes in columns.

- Measures of performance need to satisfy several criteria :
  1. They must coherently capture the aspect of performance of interest;
  2. They must be intutive enough to become widely used, so that the same measures are consistently reported by researches, enabling community-wide conclusions to be drawn;
  3. They must be computationally tractable, to match the rapid growth in scale of modem data collection.
  4. They must be simple to report as a single number for each method-dataset combination.
- Performance metrics for binary classification are designed to captured tradeoffs between four fundamental population quantities : True positives, false positives, true negatives and false negatives.
- The evaluation measures in classification problems are defined from a matrix with the numbers of examples correctly and incorrectly classified for each class, named confusion matrix. The confusion matrix for a binary classification problem is shown below.

True class	Predicted class	
	Positive	Negative
Positive	True positive	False negative
Negative	False positive	True negative

- A confusion matrix contains about actual and predicted classifications done by a classification system. Performance of such systems is commonly using data in the matrix. Confusion matrix is also called a contingency table.
  1. **False positives :** Examples predicted as positive, which are from the negative class.
  2. **False negatives :** Examples predicted as negative, whose true class is positive.
  3. **True poisitives :** Examples correctly predicted as pertaining to the positive class.
  4. **True negatives :** Examples correctly predicted as belongings to the negative class.
- The evaluation measure most used in practice is the accuracy rate. It evaluates the effectiveness of the classifier by its percentage of correct predictions.

$$\text{Accuracy rate} = \frac{|\text{True negatives}| + |\text{True positives}|}{|\text{False negatives}| + \text{False positives}| + |\text{True negatives}| + |\text{True positives}|}$$

- The complement of accuracy rate is the error rate, which evaluates a classifier by its percentage of incorrect predictions.

$$\text{Error rate} = \frac{|\text{False negatives}| + |\text{False positives}|}{|\text{False negatives}|+|\text{False positives}|+|\text{True negatives}|+|\text{True positives}|}$$

Error rate = 1 – (Accuracy rate)

- The recall and specificity measures evaluate the effectiveness of a classifier for each class in the binary problem. The recall is also known as **sensitivity** or **true positive rate**. Recall is the proportion of examples belonging to the positive class which were correctly predicted as positive.
- The specificity is a statistical measures of how well a binary classification test correctly identifies the negative cases.

$$\text{Recall (R)} = \frac{|\text{True positive}|}{||\text{True positive}|+|\text{False negative}|}$$

$$\text{Specificity} = \frac{|\text{True negative}|}{|\text{False positive}| + |\text{True positive}|}$$

Accuracy : It is a ratio of Correctly Predicted observations to total observation $\frac{TP+TN}{TP+TN+FP+FN}$

- True Positive Rate (TPR) is also called sensitivity, hit rate and recall.

$$\text{Sensitivity} = \frac{\text{Number of true positives}}{\text{Number of true positive} + \text{Number of false positive}}$$

- A statistical measure of how well a binary classification test correctly identifies a condition. Probability of correctly labeling members of the target class.
- No single measures tells the whole story. A classifier with 90 % accuracy can be useless if 90 percent of the population does not have cancer and the 10 % that do are misclassified by the classifier. Use of multiple measures recommended.

3.3.1 Accuracy and ROC Curves

ROC: True positive Rate (TPR) $= \frac{TP}{TP+FN}$

- Binary classification accuracy metrics quantify the two types of correct predictions and two types of errors. Typical metrics are accuracy (ACC), precision, recall, false positive rate, F1-measure. Each metric measures a different aspect of the predicative model.

False Positive Rate (FPR) $= \frac{FP}{FP+TN}$

- Accuracy (ACC) measures the fraction of correct predictions. Precision measures the fraction of actual positives among those examples that are predicted as positive. Recall measures how many actual positives were predicted as positive. F1-measure is the harmonic mean of precision and recall.

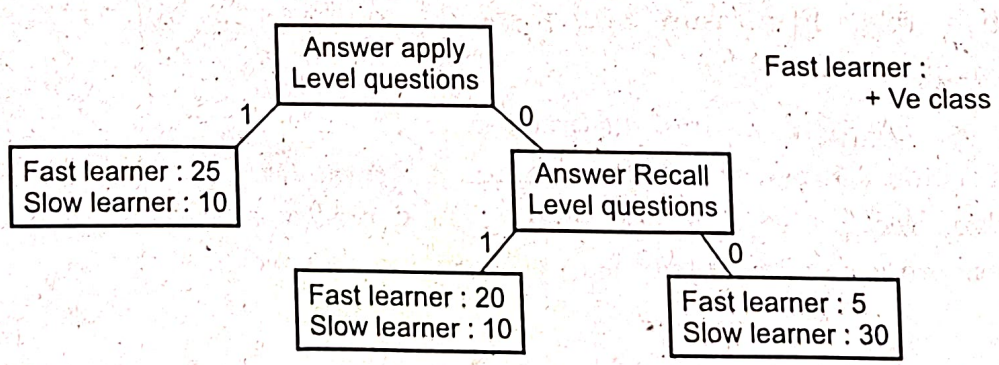
ROC curve

- Receiver Operating Characteristics (ROC) graphs have long been used in signal detection theory to depict the tradeoff between hit rates and false alarm fates over

noisy channel. Recent years have seen an increase in the use of ROC graphs in the machine learning community.

- An ROC plot plots true positive rate on the Y-axis false positive rate on the X-axis; a single contingency table corresponds to a single point in an ROC plot.
- The performance of a ranker can be assessed by drawing a piecewise linear curve in an ROC plot, known as an ROC curve. The curve starts in (0, 0), finishes in (1, 1) and is monitorically non-decreasing in both axes.
- A useful technique for organizing classifiers and visualizing their performance. Especially useful for domains with skewed class distribution and unequal classification error costs.
- It allows to create ROC curve and a complete sensitivity/specificity report. The ROC curve is a fundamental tool for diagnostic test evaluation.
- In a ROC curve the true positive rate (Sensitivity) is plotted in function of the false positive rate (100 Specificity) for different cut-off points of a parameter. Each point on the ROC curve represents a sensitivity/specificity pair corresponding to a particular decision threshold. The area under the ROC curve is a measure of how well a parameter can distinguish between two diagnostic groups.
- Each point on an ROC curve connecting two segments corresponds to the true and false positive rates achieved on the same test set by the classifier obtained from the ranker by splitting the ranking between those two segments.
- An ROC curve is convex if the slopes are montonically non-increasing when moving along the curve from (0, 0) to (1, 1). A concavity in an ROC curve, i.e., two or more adjacent segments with increasing slopes, indicates a locally worse than random ranking. In this we would get better ranking performance by joining the segments involved in the concavity, thus creating a coarser classifier.

Example 3.3.1 *i) Find contingency table ii) Find recall iii) Precision iv) Negative recall v) False positive rate*



Solution : Contingency table

	Predicted			Total
Faster Learner	25	20	5	50
Slow Learner	10	10	30	50
Total	35	30	35	100

Actual

$$\text{Precision} = \frac{\text{True positive}}{\text{Actual results}} \quad \text{or} \quad \frac{\text{True positive}}{\text{True positive} + \text{False positive}}$$

$$\text{Recall} = \frac{\text{True positive}}{\text{Predicted results}} \quad \text{or} \quad \frac{\text{True positive}}{\text{True positive} + \text{False negative}}$$

Calculate precision and recall

Precision = 25/35 = 0.714

Recall = 25/30 = 0.833

False positive rate = (False positive) / (False positive + True negative)

= 10/(10 + 30) = **0.25**

Example 3.3.2 *Consider following confusion matrix and calculate following i) Sensitivity of classifier ii) Specificity of classifier.*

Confusion matrix		Predicted		Total
Actual	+	8	10	18
	−	4	8	12
Total		12	18	30

Solution : Given data : TP = 8, FN = 10, FP = 4, TN = 8

- Sensitivity (SN) is calculated as the number of correct positive predictions divided by the total number of positives. It is also called recall (REC) or True Positive Rate (TPR)

$$\text{Sensitivity (SN)} = \frac{TP}{TP+FN} = \frac{8}{8+10} = \mathbf{0.444}.$$

- Specificity (SP) is calculated as the number of correct negative predictions divided by the total number of negatives. It is also called True Negative Rate (TNR).

$$\text{Specificity (SP)} = \frac{TN}{TN+FP} = \frac{8}{8+4} = \mathbf{0.666}$$

Example 3.3.3 *Consider the following 3-class confusion matrix. Calculate precision and recall per class. Also calculate weighted average precision and recall for classifier.*

	Predicted		
Actual	15	2	3
	7	15	8
	2	3	45

Solution :

	Predicted			
Actual	15	2	3	20
	7	15	8	30
	2	3	45	50
	24	20	56	100

Classifier accuracy = $\frac{15+15+45}{100} = \frac{75}{100} = 0.75$

Calculate per-class precision and recall :

First class = $\frac{15}{24} = 0.63$ and $\frac{15}{20} = 0.75$

Second class = $\frac{15}{20} = 0.75$ and $\frac{15}{30} = 0.50$

Third class = $\frac{45}{56} = 0.8$ and $\frac{45}{50} = 0.9$

Example 3.3.4 *Prove that : i) FPR = 1 – TPR ii) FNR = 1 – TPR.*

Solution : i) FPR = 1–TPR

False Positive Rate (FPR) = 1 – True Negative Rate (TNR)

FPR = FP/N = FP/ (FP + TN)

FPR = 1 – TNR

ii) FNR = 1 – TPR

False Negative Rate = 1 – True Positive Rate

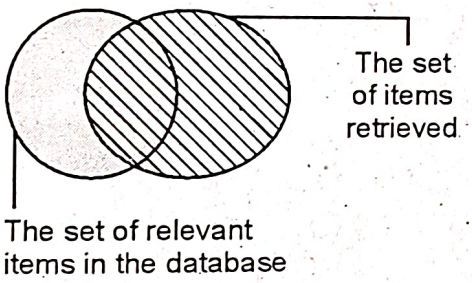
FNR = FN (FN + TP)

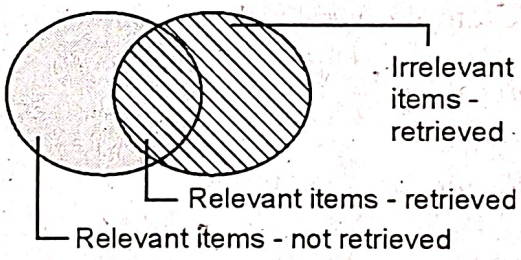
FNR = 1 – TPR

3.3.2 Precision and Recall

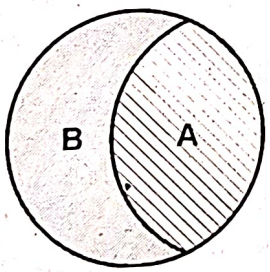
- **Relevance :** Relevance is a subjective notion. Different users may differ about the relevance or non-relevance of particular documents to given questions.
- In response to a query, an IR system searches its document collection and returns a ordered list of responses. It is called the retrieved set or ranked list. The system employs a search strategy or algorithm and measure the quality of a ranked list.
- A better search strategy yields a better ranked list and better ranked lists help the user fill their information need.
- Precision and recall are the basic measures used in evaluating search strategies. As shown in the first two figures, these measures assume :

1. There is a set of records in the database which is relevant to the search topic
2. Records are assumed to be either relevant or irrelevant.
3. The actual retrieval set may not perfectly match the set of relevant records.





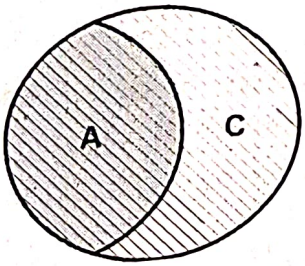
- **Recall** is the ratio of the number of relevant records retrieved to the total number of relevant records in the database. It is usually expressed as a percentage.



A = Number of relevant records retrieved.
B = Number of relevant records not retrieved.

$$\text{Recall} = \frac{A}{A+B} \times 100\ \%$$

Precision is the ratio of the number of relevant records retrieved to the total number of irrelevant and relevant records retrieved. It is usually expressed as a percentage.



A = Number of relevant records retrieved.
C = Number of irrelevant records retrieved.

$$\text{Precison} = \frac{A}{A+C} \times 100\ \%$$

- As recall increases, the precision decreases and recall decreases the precision increases.

Example 3.3.5 *Assume the following :*

A database contains 80 records on a particular topic

A search was conducted on that topic and 60 records were retrieved.

Of the 60 records retrieved, 45 were relevant.

Calculate the precision and recall scores for the search.

Solution : Using the designations above :

A = The number of relevant records retrieved,

B = The number of relevant records not retrieved and

C = The number of irrelevant records retrieved.

In this example A = 45, B = 35 (80 – 45) and C = 15 (60 – 45).

$$\text{Recall} = \frac{45}{45+35} \times 100\ \%$$

$$\text{Recall} = \frac{45}{80} \times 100\ \%$$

$$\textbf{Recall} = \textbf{56.25 \%}$$

$$\text{Precision} = \frac{A}{A+C} \times 100\ \%$$

$$\text{Precision} = \frac{45}{45+15} \times 100\ \% = \frac{45}{60} \times 100$$

$$\textbf{Precision} = \textbf{75 \%}$$

Example 3.3.6 *20 found documents, 18 relevant, 3 relevant documents are not found, 27 irrelevant are as well not found. Calculate the precision and recall and fallout scores for the search.*

Solution :

Precision : 18/20 = 90 %

Recall : 18/21 = 85.7 %

Fall-out : 2/29 = 6.9 %

- Recall is a non-decreasing function of the number of docs retrieved. In a good system, precision decreases as either the number of docs retrieved or recall increases. This is not a theorem, but a result with strong empirical confirmation.

- The set of ordered pairs makes up the precision-recall graph. Geometrically when the points have been joined up in some way they make up the precision-recall curve. The performance of each request is usually given by a precision-recall curve. To measure the overall performance of a system, the set of curves, one for each request, is combined in some way to produce an average curve.
- Assume that set R_q containing the relevant document for q has been defined. Without loss of generality, assume further that the set R_q is composed of the following documents :

$$R_q = \{ d_3, d_5, d_9, d_{25}, d_{39}, d_{44}, d_{56}, d_{71}, d_{89}, d_{123} \}$$

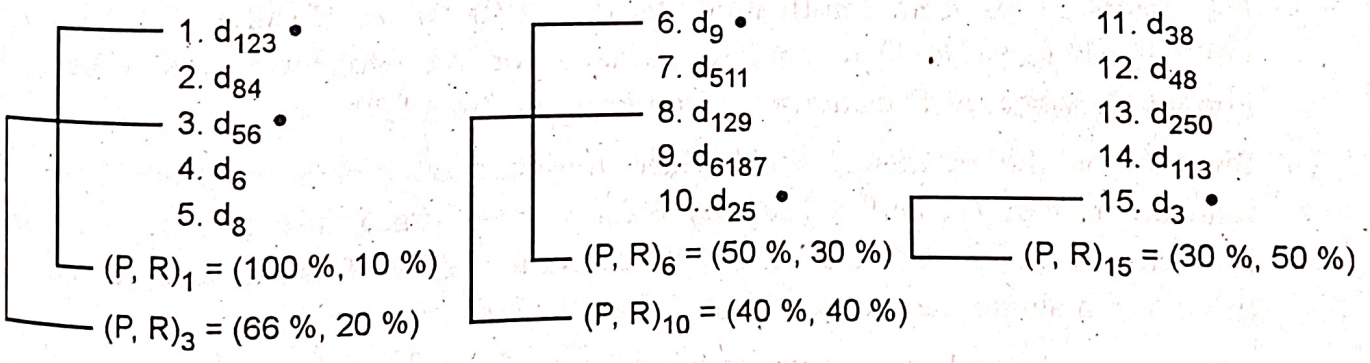
There are ten documents which are relevant to the query q.

- For the query q, a ranking of the documents in the answer set as follows.

Ranking for query q :

1. d_{123} *	6. d_9 *	11. d_{38}
2. d_{84}	7. d_{511}	12. d_{48}
3. d_{56} *	8. d_{129}	13. d_{250}
4. d_6	9. d_{187}	14. d_{113}
5. d_8	10. d_{25} *	15. d_3 *

- The documents that are relevant to the query q are marked with star after the



document number. Ten relevant documents, five included in Top 15 .

- Fig 3.3.1 shows the curve of precision versus recall. By taking various numbers of the top returned documents (levels of recall), the evaluator can produce a precision-recall curve.
- The precision versus recall curve is usually plotted based on 11 standard recall level: 0 %,10 %,....,100 %.

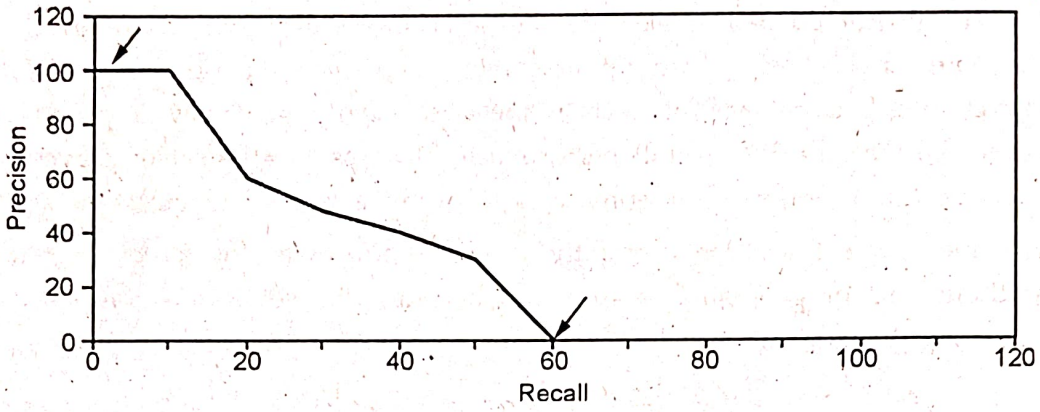


Fig. 3.3.1 Precision versus recall curve

- In this example : The precisions for recall levels higher than 50 % drop to 0 because no relevant documents were retrieved. There was an interpolation for the recall level 0 %.
- Since the recall levels for each query might be distinct from the 11 standard recall levels.

3.3.3 F - Measure

- The F measure is a measure of a test's accuracy and is defined as the weighted harmonic mean of the precision and recall of the test. The F - measure or F - score is one of the most commonly used "single number" measures in Information Retrieval, Natural Language Processing and Machine Learning.
- F-measure comes from Information Retrieval (IR) where Recall is the frequency with which relevant documents are retrieved or 'recalled' by a system, but it is known elsewhere as Sensitivity or True Positive Rate (TPR).
- Precision is the frequency with which retrieved documents or predictions are relevant or 'correct', and is properly a form of Accuracy, also known as Positive Predictive Value (PPV) or True Positive Accuracy (TPA). F is intended to combine these into a single measure of search 'effectiveness'.
- High precision and low accuracy is possible due to systematic bias. One of the problems with Recall, Precision, F - measure and Accuracy as used in Information Retrieval is that they are easily biased.
- The F-measure balances the precision and recall. The result is a value between 0.0 for the worst F-measure and 1.0 for a perfect F - measure.
- The formula for the standard F1 - score is the harmonic mean of the precision and recall. A perfect model has an F-score of 1.

$$\text{F - Measure} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Review Questions

1. *Define following terms with suitable example :*
   i) Confusion matrix ii) False positive rate iii) True positive rate.
2. *What is a contingency table/matrix ? What is the use of it ?*
3. *Explain true positive, true negative false positives, false negatives and class ratio.*
4. *What is a contingency table ? What does it represent ?*
5. *What is multiple linear regression ? How will it be different from simple linear regression ?*

3.4 Multiclass Classification

- Multiclass classification is a machine learning classification task that consists of more than two classes, or outputs. For example, using a model to identify animal types in images from an encyclopedia is a multiclass classification example because there are many different animal classifications that each image can be classified as. Multiclass classification also requires that a sample only have one class.
- Each training point belongs to one of N different classes. The goal is to construct a function which, given a new data point, will correctly predict the class to which the new point belongs.
- There are many scenarios in which there are multiple categories to which points belong, but a given point can belong to multiple categories. In its most basic form, this problem decomposes trivially into a set of unlinked binary problems, which can be solved naturally using techniques for binary classification.
- Common model for classification is the Support Vector Machine (SVM). An SVM works by projecting the data into a higher dimensional space and separating it into different classes by using a single (or set of) hyperplanes. A single SVM does binary classification and can differentiate between two classes. In order to differentiate between K classes, one can use (K – 1) SVMs. Each one would predict membership in one of the K classes.

3.4.1 Weighted Average

- **Mean Average Precision (MAP)** is also called average precision at seen relevant documents. It determine precision at each point when a new relevant document gets retrieved. Average of the precision value obtained for the top k documents, each time a relevant doc is retrieved.

- Avoids interpolation, use of fixed recall levels. MAP for query collection is arithmetic averaging. Average precision - recall curves are normally used to compare the performance of distinct IR algorithms.
- Use P = 0 for each relevant document that was not retrieved. Determine average for each query, then average over queries :

$$MAP = \frac{1}{N}\sum_{j=1}^{N}\frac{1}{Q_j}\sum_{i=1}^{Q_j}P(doc_i)$$

where Q_j = Number of relevant document for query j

N = Number of queries

$P(doc_i)$ = Precision at i^{th} relevant document

Precision - recall appropriateness :

- Precision and recall have been extensively used to evaluate the retrieval performance of IR algorithms. However, a more careful reflection reveals problems with these two measures :
- First, the proper estimation of maximum recall for a query requires detailed knowledge of all the documents in the collection.
- Second, in many situations the use of a single measure could be more appropriate.
- Third, recall and precision measure the effectiveness over a set of queries processed in batch mode.
- Fourth, for systems which require a weak ordering though, recall and precision might be inadequate.

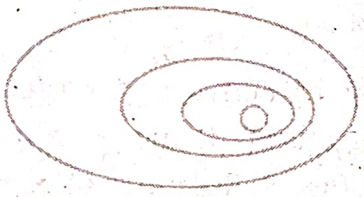
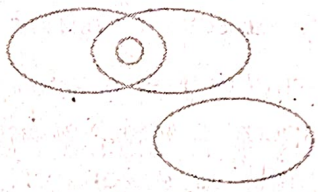
Single value summaries :

- Average precision - Recall curves constitute standard evaluation metrics for information retrieval systems. However, there are situations in which we would like to evaluate retrieval performance over individual queries. The reasons are twofold :
  1. First, averaging precision over many queries might disguise important anomalies in the retrieval algorithms under study.
  2. Second, we might be interested in investigating whether a algorithm outperforms the other for each query.
- In these situations, a single precision value can be used.

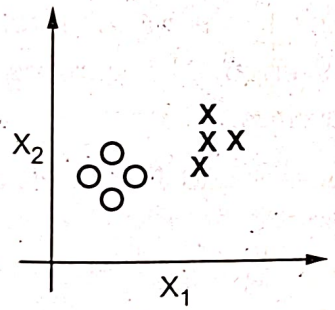
3.4.2 Multiclass Classification Techniques

- Each training point belongs to one of N different classes. The goal is to construct a function which, given a new data point, will correctly predict the class to which the new point belongs. The multi-class classification problem refers to assigning each of the observations into one of k classes.

- A common way to combine pair wise comparisons is by voting. It constructs a rule for discriminating between every pair of classes and then selecting the class with the most winning two-class decisions. Though the voting procedure requires just pair wise decisions, it only predicts a class label.
- Example of Multi-label classification is as follows :

1. Is it eatable ? 2. Is it sweet ? 3. Is it a fruit ? 4. Is it a banana ?	1. Is it a banana ? 2. Is it an apple ? 3. Is it an orange ? 4. Is it a pineapple ?	1. Is it a banana ? 2. Is it yellow ? 3. Is it sweet ? 4. Is it round ?
		
Nested/Hierarchical	Exclusive/Multi-class	General/Structured

- Fig. 3.4.1 and 3.4.2 shows binary and multiclass classification.
- Multiclass classification through binary classification :



- **Fig. 3.4.1 Binary classification**

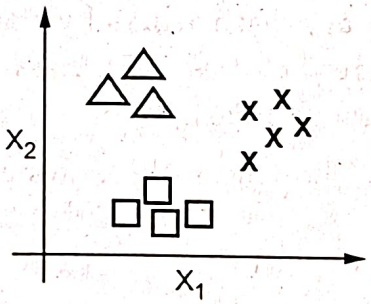


Fig. 3.4.2 Multiclass classification

1. One Vs All (OVA) :

- For each class build a classifier for that class vs the rest. Build N different binary classifiers.
- For this approach, we require N = K binary classifiers, where the k^{th} classifier is trained with positive examples belonging to class k and negative examples belonging to the other K – 1 classes.
- When testing an unknown example, the classifier producing the maximum output is considered the winner, and this class label is assigned to that example.
- It is simple and provides performance that is comparable to other more complicated approaches when the binary classifier is tuned well.

2. All-Vs-All (AVA) :

- For each class build a classifier for those class vs the rest. Build N (N – 1) classifiers, one classifier to distinguish each pair of classes i and j .
- A binary classifier is built to discriminate between each pair of classes, while discarding the rest of the classes.
- When testing a new example, a voting is performed among the classifiers and the class with the maximum number of votes wins.

3. Calibration

- The decision function f of a classifier is said to be calibrated or well-calibrated if P (x is correctly classified | f(x) = s) ≃ s
- Informally f is a good estimate of the probability of classifying correctly a new datapoint x which would have output value x. Intuitively if the "raw" output of a classifier is g you can calibrate it by estimating the probability of x being well classified given that g(x)=y for all y values possible.

4. Error-Correcting Output-Coding (ECOC)

- Error correcting code approaches try to combine binary classifiers in a way that lets you exploit de-correlations and correct errors.
- This approach works by training N binary classifiers to distinguish between the K different classes. Each class is given a codeword of length N according to a binary matrix M . Each row of M corresponds to a certain class.
- The following table shows an example for K = 5 classes and N = 7 bit code words.

	f_1	f_2	f_3	f_4	f_5	f_6	f_7
Class 1	0	0	0	0	0	0	0
Class 2	0	1	1	0	0	1	1
Class 3	0	1	1	1	1	0	0
Class 4	1	0	1	1	0	1	0
Class 5	1	1	0	1	0	0	1

- Each class is given a row of the matrix. Each column is used to train a distinct binary classifier. When testing an unseen example, the output codeword from the N classifiers is compared to the given K code words, and the one with the minimum hamming distance is considered the class label for that example.

Example 3.4.1 *Using following feature tree, write decision rules for majority class.*

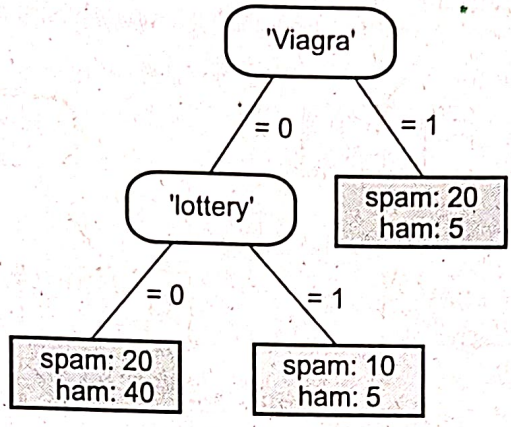


Fig. 3.4.3

Solution : Left side : A feature tree combining two Boolean features. Each internal node or split is labelled with a feature, and each edge emanating from a split is labelled with a feature value. Each leaf therefore corresponds to a unique combination of feature values. Also indicated in each leaf is the class distribution derived from the training set.

- **Right side :** A feature tree partitions the instance space into rectangular regions, one for each leaf.

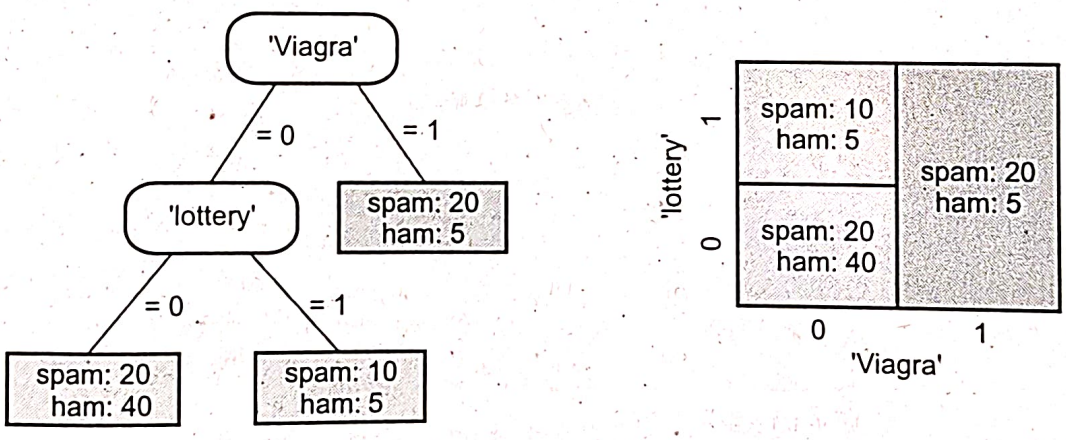


Fig. 3.4.4

- The leaves of the tree in the above figure could be labelled, from left to right, as ham - spam - spam, employing a simple decision rule called majority class.
- **Left side :** A feature tree with training set class distribution in the leaves.
- **Right side :** A decision tree obtained using the majority class decision rule.

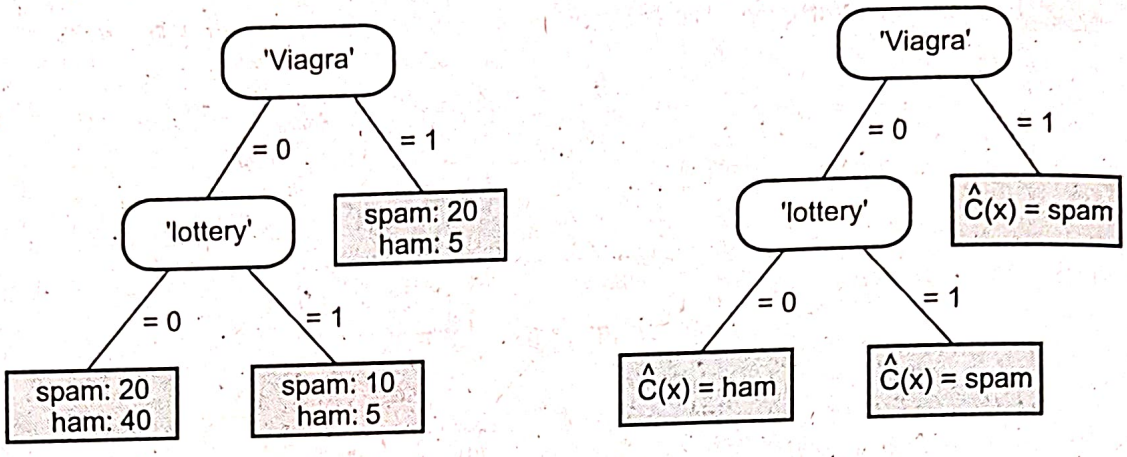


Fig. 3.4.5

Example 3.4.2 *Consider the following three-class confusion matrix.*

		Predicted		
Actual	15	2	3
	7	15	8
	2	3	45

Calculate precision and recall per class. Also calculate weighted average precision and recall for the classifier.

Solution :

		Predicted		
Actual	15	2	3	20
	7	15	8	30
	2	3	45	50
	24	20	56	100

Classifier accuracy $= \frac{15+15+45}{100} = \frac{75}{100} = 0.75$

Calculate per-class precision and recall :

First class $= \frac{15}{24} = 0.63$ and $\frac{15}{20} = 0.75$

Second class $= \frac{15}{20} = 0.75$ and $\frac{15}{30} = 0.50$

Third class $= \frac{45}{56} = 0.8$ and $\frac{45}{50} = 0.9$

Example 3.4.3 *Prove with an example* ***Accuracy = 1 – error rate.***

Solution : Accuracy is the percent of correct classifications. Error rate is the percent of incorrect classifications. Classification accuracy is a misleading measure of performance when the data are not perfectly balanced. This is because a classifier may take advantage of an imbalanced dataset and trivially achieve a classification accuracy equal to the fraction of the majority class.

Review Questions

1. *Explain construction of multi-class classifier,*
   i) One Vs all approach ii) One Vs one approach
   iii) Error correcting output codes approach.
2. *Explain any two approaches to construct multiclass classifier.*

3.5 Performance Evaluation : Micro-average Method

- In micro-average technique, you sum up the individual proper positives, fake positives and false negatives of the gadget for exceptional units and then apply them to get the statistics. For example, for a set of information, the system's
  True advantageous (TP1) = 12
  False nice (FP1) = 9
  False terrible (FN1) = three
- Then precision (P1) and recollect (R1) could be 57.14 and 80 and for a one-of-a-kind set of information, the gadget's
  True effective (TP2) = 50
  False fine (FP2) = 23
  False bad (FN2) = 9
- Then precision (P2) and do not forget (R2) will be sixty eight.49 and 84.Seventy five.
- Now, the common precision and consider of the machine the use of the Micro-common approach is

Micro-average of precision = (TP1+TP2)/(TP1+TP2+FP1+FP2)

= (12+50)/(12+50+9+23) = 65.Ninety six

Micro-average of don't forget = (TP1+TP2)/(TP1+TP2+FN1+FN2)

= (12+50)/(12+50+Three+9) = Eighty three.Seventy eight

- The micro-common F-score might be simply the harmonic imply of those figures.
- Eg-In micro-average approach, you sum up the man or woman proper positives, fake positives and fake negatives of the system for exceptional sets and the practice them to get the facts. For example, for a set of statistics, then device's

True tremendous (TP1) = 12

False effective (FP1) = Nine

False terrible (FN1) = Three

- Then precision (P1) and bear in mind (R1) could be fifty seven.14 and eighty and for a special set of information, the gadget's

True tremendous (TP2) = 50

False positive (FP2) = 23

False bad (FN2) = Nine

- Then precision (P2) and recollect (R2) can be 68.49 and eighty four.Seventy five.
- Now, the common precision and recall of the system the use of the micro-average technique is

Micro-average of precision = (TP1+TP2)/(TP1+TP2+FP1+FP2)

= (12+50)/(12+50+9+23) = 65.Ninety six

Micro-common of consider = (TP1+TP2)/(TP1+TP2+FN1+FN2)

= (12+50)/(12+50+Three+Nine) = 83.Seventy eight

- The micro-average F-score can be virtually the harmonic imply of those two figures.

3.5.1 Marco-average Method

- Macro-average approach can be used whilst you need to know how the machine performs usually across the units of facts. You have to no longer give you any particular choice with this average.
- On the other hand, micro-common may be a useful measure whilst your dataset varies in length.
- This method is simple. Just take the common of the precision and don't forget the system on specific sets. For example, the macro-average precision and don't forget of the gadget for the given example is,

Macro-average precision = (P1+P2)/2 = (57.14+68.Forty nine)/2 = Sixty two. Eighty two

Macro-average don't forget = (R1+R2)/2 = (80+84.75)/2 = Eighty two.25

3.5.1.1 Suitability

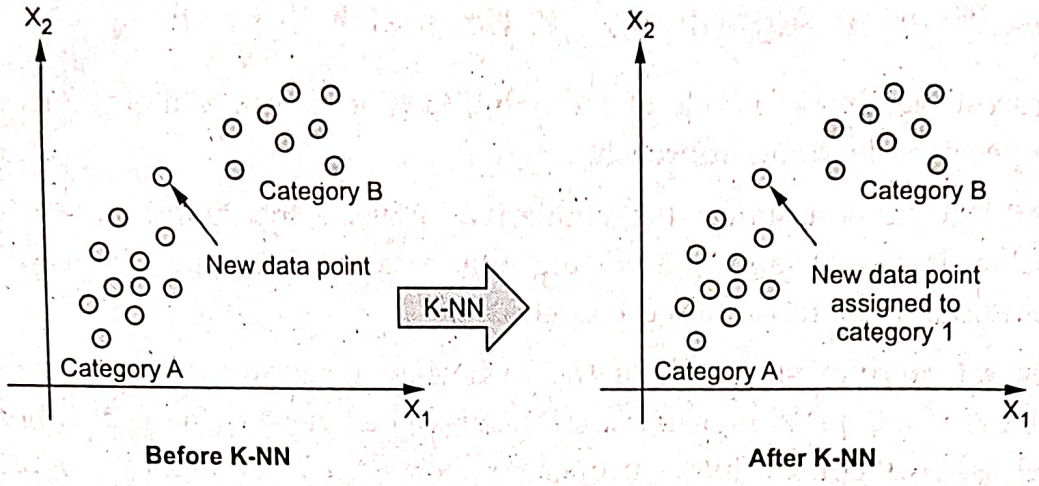
- Macro-average approach can be used when you want to recognise how the gadget plays ordinary across the sets of information. You must no longer provide you with any unique decision with this common.
- On the opposite hand, micro-common can be a useful measure while your dataset varies in size.

3.6 Classification Algorithms : K Nearest Neighbor

- K-Nearest Neighbour is one of the only Machine Learning algorithms based totally on supervised learning approach.
- K-NN algorithm assumes the similarity between the brand new case/facts and available instances and placed the brand new case into the category that is maximum similar to the to be had classes.
- K-NN set of rules shops all of the to be had facts and classifies a new statistics point based at the similarity. This means when new data seems then it may be effortlessly categorised into a properly suite class by using K-NN algorithm.
- K-NN set of rules can be used for regression as well as for classification however normally it's miles used for the classification troubles.
- K-NN is a non-parametric algorithm, because of this it does no longer makes any assumption on underlying data.
- It is also referred to as a lazy learner set of rules because it does no longer research from the training set immediately as a substitute it shops the dataset and at the time of class, it plays an movement at the dataset.
- The KNN set of rules at the schooling section simply stores the dataset and when it gets new data, then it classifies that statistics into a class that is an awful lot similar to the brand new data.
- Example : Suppose, we've an picture of a creature that looks much like cat and dog, but we want to know both it is a cat or dog. So for this identity, we are able to use the KNN algorithm, because it works on a similarity degree. Our KNN version will discover the similar features of the new facts set to the cats and dogs snap shots and primarily based on the most similar functions it will place it in both cat or canine class.

3.6.1 Why Do We Need KNN ?

- Suppose there are two categories, i.e., category A and category B and we've a brand new statistics point x1, so this fact point will lie within of these classes. To solve this sort of problem, we need a K-NN set of rules. With the help of K-NN, we will without difficulty discover the category or class of a selected dataset. Consider the underneath diagram :



Before K-NN After K-NN

Fig. 3.6.1 Why do we need KNN ?

3.6.2 How Does KNN Work ?

- The K-NN working can be explained on the basis of the below algorithm :

Step - 1 : Select the wide variety K of the acquaintances.

Step - 2 : Calculate the Euclidean distance of K variety of friends.

Step - 3 : Take the K nearest neighbors as according to the calculated Euclidean distance.

Step - 4 : Among these ok pals, count number the number of the data points in each class.

Step - 5 : Assign the brand new records points to that category for which the quantity of the neighbor is maximum.

Step - 6 : Our model is ready.

- Suppose we've got a brand new information point and we want to place it in the required category. Consider the under image

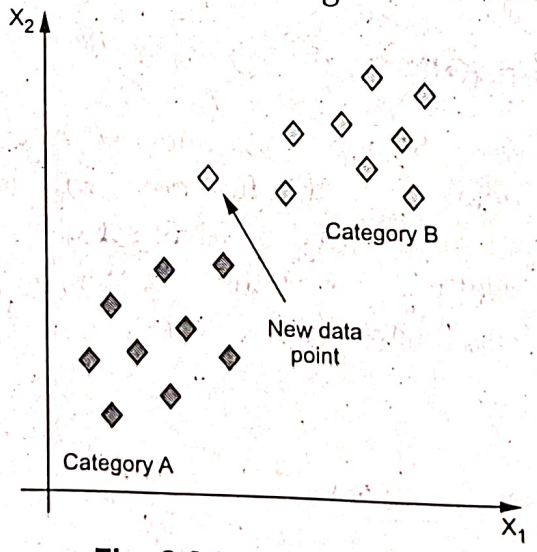
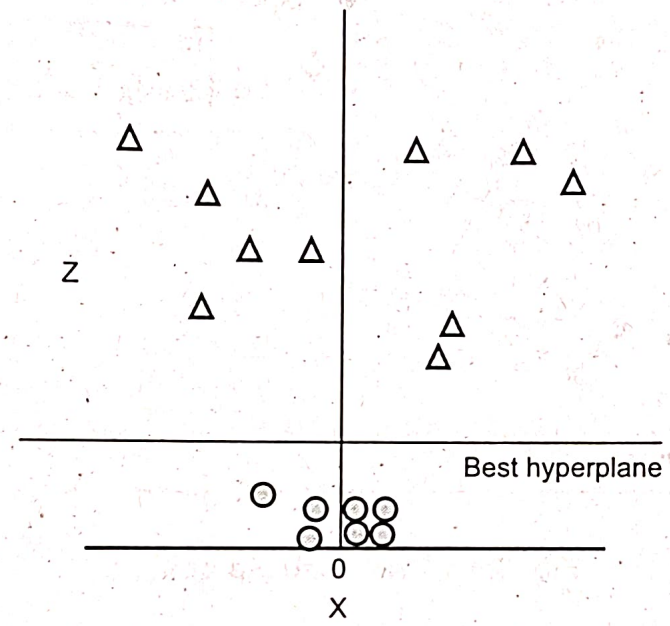
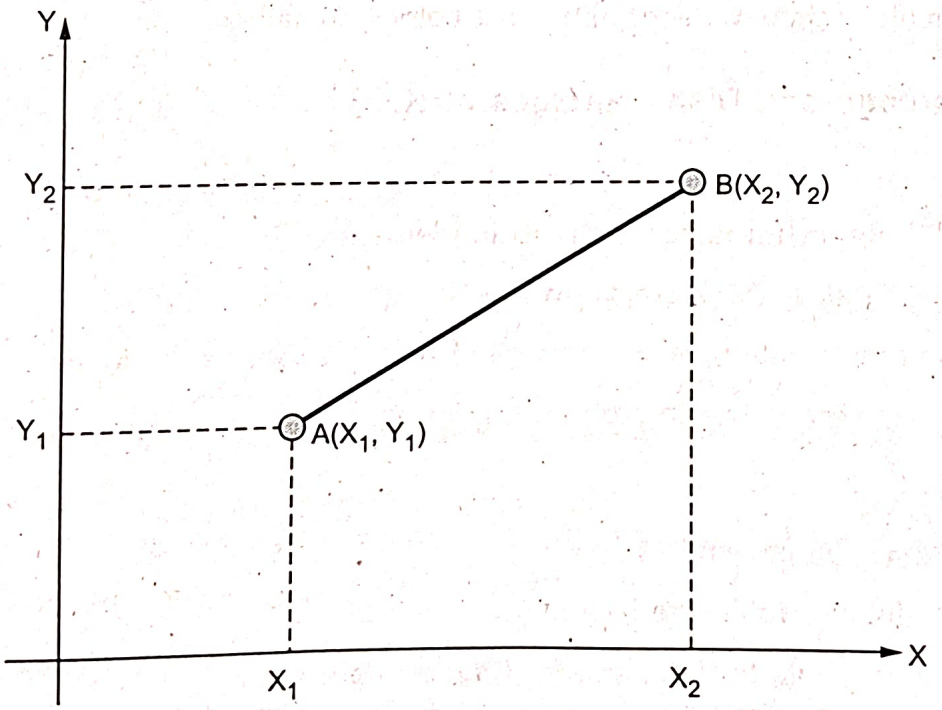


Fig. 3.6.2 KNN example

- Firstly, we are able to pick the number of friends, so we are able to select the ok = 5.
- Next, we will calculate the Euclidean distance between the facts points. The Euclidean distance is the gap between points, which we've got already studied in geometry. It may be calculated as :





Euclidean distance between A_1 and B_2 = $\sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$

Fig. 3.6.3 KNN example continue

- By calculating the Euclidean distance we got the nearest acquaintances, as 3 nearest neighbours in category A and two nearest associates in class B. Consider the underneath image.

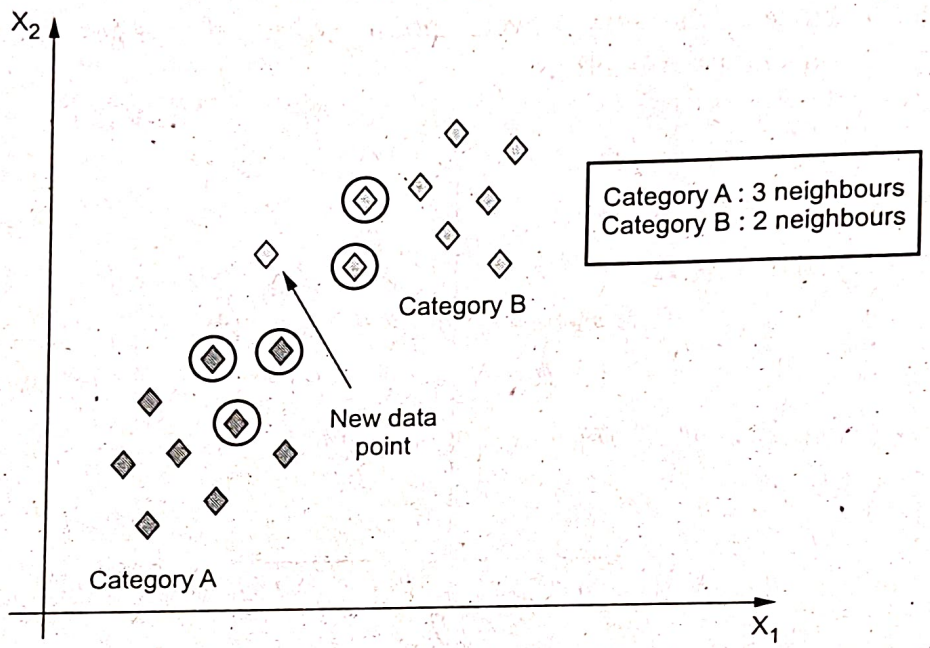


Fig. 3.6.4 KNN example continue

- As we are able to see the three nearest acquaintances are from category A, subsequently this new fact point must belong to category A.

3.6.3 Advantage and Disadvantages of KNN

Advantages :

1. The KNN algorithm is very easy to implement.
2. Nearly optimal in the large sample limit.
3. Uses local information, which can yield highly adaptive behavior.
4. Lends itself very easily to parallel implementations.

Disadvantages :

1. Large storage requirements.
2. Computationally intensive recall.
3. Highly susceptible to the curse of dimensionality.

Review Questions

1. *What is the KNN algorithm ? Explain in detail.*
2. *Why do you need KNN ? How it works is explained with an example.*

3.7 Linear Support Vector Machines (SVM)

- Support Vector Machines (SVMs) are a set of supervised learning methods which learn from the dataset and used for classification. SVM is a classifier derived from statistical learning theory by Vapnik and Chervonenkis.

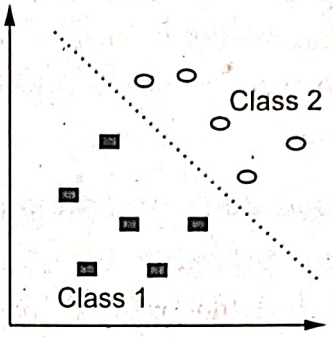


Fig. 3.7.1 Two class problem

- An SVM is a kind of large-margin classifier: it is a vector space based machine learning method where the goal is to find a decision boundary between two classes that is maximally far from any point in the training data
- Given a set of training examples, each marked as belonging to one of two classes, an SVM algorithm builds a model that predicts whether a new example falls into one class or the other. Simply speaking, we can think of an SVM model as representing the examples as points in space, mapped so that each of the examples of the separate classes are divided by a gap that is as wide as possible.

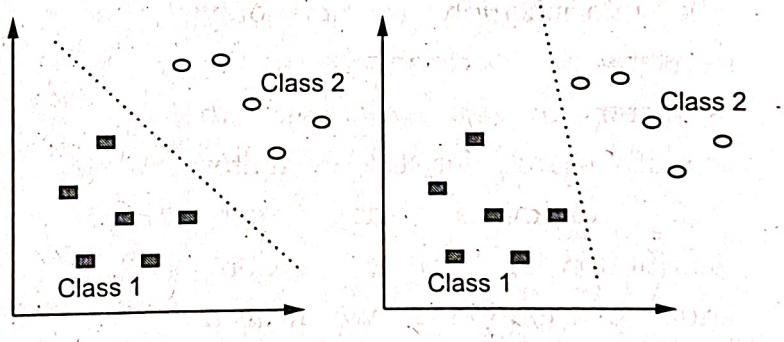


Fig. 3.7.2 Bad decision boundary of SVM

- New examples are then mapped into the same space and classified to belong to the class based on which side of the gap they fall on.

Two class problems

- Many decision boundaries can separate these two classes. Which one should we choose ?
- Perceptron learning rule can be used to find any decision boundary between class 1 and class 2.

- The line that maximizes the minimum margin is a good bet. The model class of "hyper-planes with a margin of m" has a low VC dimension if m is big.
- This maximum-margin separator is determined by a subset of the data points. Data points in this subset are called **"support vectors"**. It will be useful computationally if only a small fraction of the data points are support vectors, because we use the support vectors to decide which side of the separator a test case is on.

Example of bad decision boundaries

- SVM are primarily two-class classifiers with the distinct characteristic that they aim to find the optimal hyperplane such that the expected generalization error is minimized. Instead of directly minimizing the empirical risk calculated from the training data, SVMs perform structural risk minimization to achieve good generalization.
- The empirical risk is the average loss of an estimator for a finite set of data drawn from P. The idea of risk minimization is not only measure the performance of an estimator by its risk, but to actually search for the estimator that minimizes risk over distribution P. Because we don't know distribution P we instead minimize empirical risk over a training dataset drawn from P. This general learning technique is called **empirical risk minimization.**

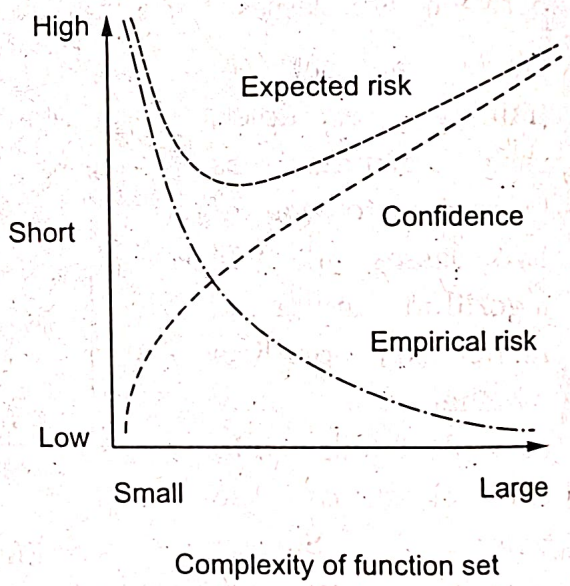


Fig. 3.7.3 Empirical risk

- Fig. 3.7.3 shows empirical risk.

Good decision boundary : Margin should be large

- The decision boundary should be as far away from the data of both classes as possible. If data points lie very close to the boundary, the classifier may be consistent but is more "likely" to make errors on new instances from the distribution. Hence, we prefer classifiers that maximize the minimal distance of data points to the separator.

1. **Margin (m)** : The gap between data points and the classifier boundary. The margin is the minimum distance of any sample to the decision boundary. If this hyperplane is in the canonical form, the margin can be measured by the length of the weight vector. The margin is given by the projection of the distance between these two points on the direction perpendicular to the hyperplane. **Margin** of the separator is the distance between support vectors.

$$\text{Margin (m)} = \frac{2}{||w||}$$

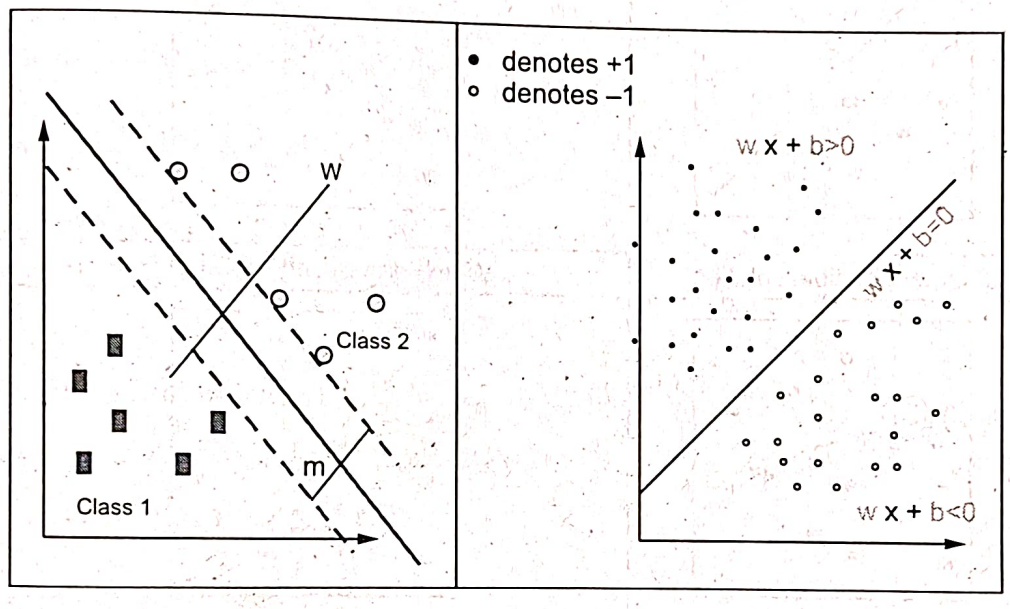


Fig. 3.7.4 Good decision boundary

2. **Maximal margin classifier** : A classifier in the family F that maximizes the margin. Maximizing the margin is good according to intuition and PAC theory. Implies that only support vectors matter; other training examples are ignorable.

Example 3.7.1 *For the following figure, find a linear hyperplane (decision boundary) that will separate the data.*

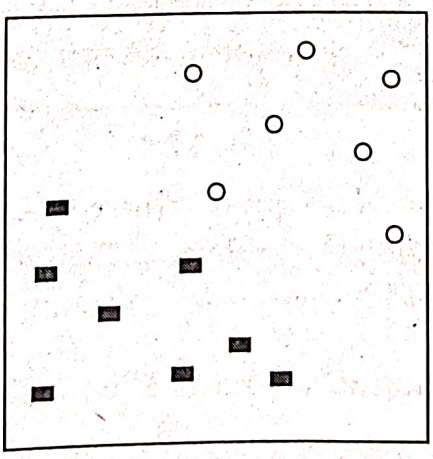


Fig. 3.7.5

Solution :

1. Define what an optimal hyperplane is : **Maximize margin**
2. Extend the above definition for non-linearly separable problems : **Have a penalty term for misclassifications.**

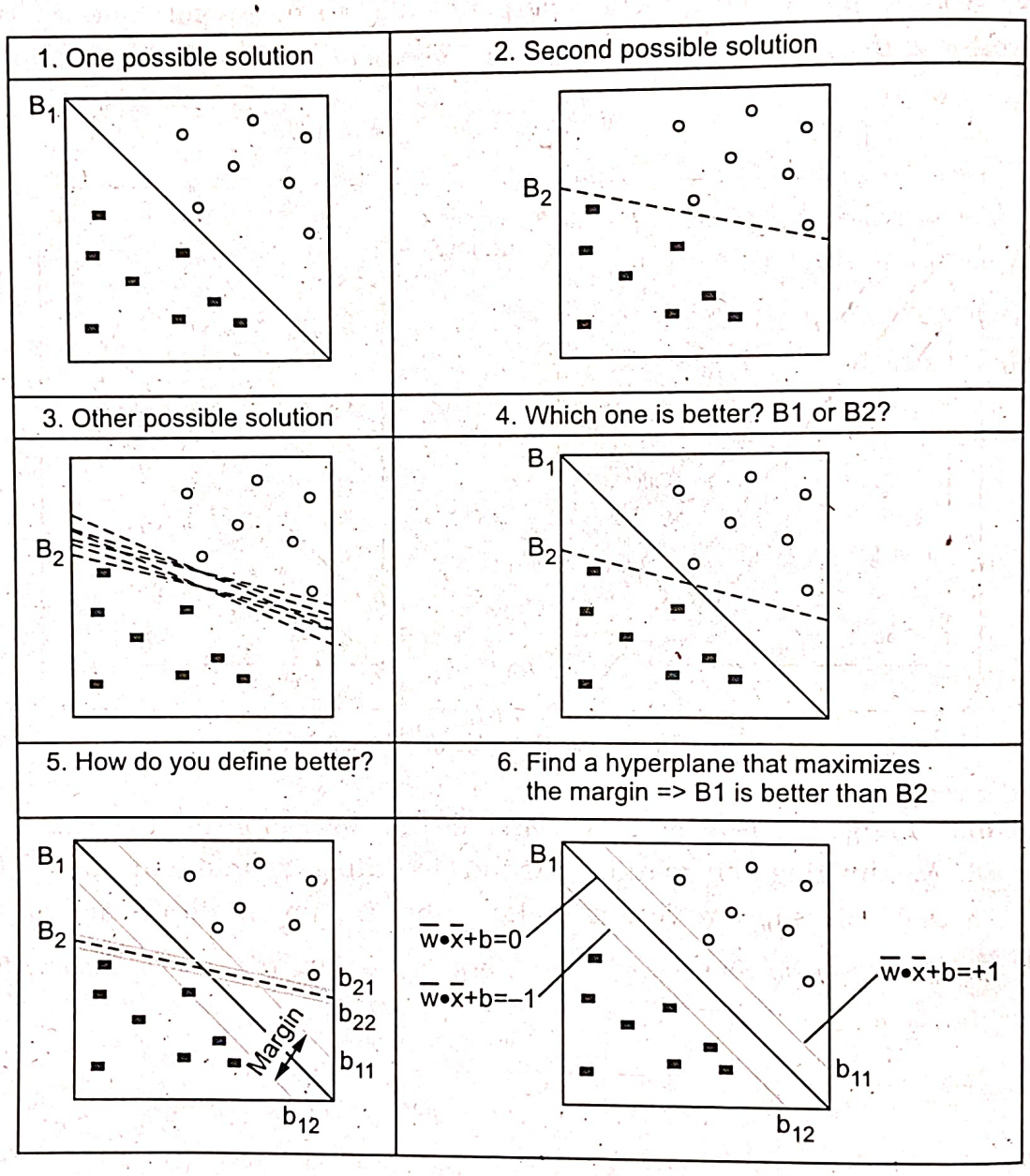


Fig. 3.7.6

3. Map data to high dimensional space where it is easier to classify with linear decision surfaces : **reformulate problem so that data is mapped implicitly to this space.**

3.7.1 Key Properties of Support Vector Machines

1. Use a single hyperplane which subdivides the space into two half-spaces, one which is occupied by Class 1 and the other by Class 2.

2. They maximize the margin of the decision boundary using quadratic optimization techniques which find the optimal hyperplane.
3. Ability to handle large feature spaces.
4. Overfitting can be controlled by soft margin approach.
5. When used in practice, SVM approaches frequently map the examples to a higher dimensional space and find margin maximal hyperplanes in the mapped space, obtaining decision boundaries which are not hyperplanes in the original space.
6. The most popular versions of SVMs use non-linear kernel functions and map the attribute space into a higher dimensional space to facilitate finding "good" linear decision boundaries in the modified space.

3.7.2 SVM Applications

- SVM has been used successfully in many real-world problems,
  1. Text (and hypertext) categorization.
  2. Image classification.
  3. Bioinformatics (Protein classification, Cancer classification).
  4. Hand-written character recognition.
  5. Determination of SPAM email.

3.7.3 Limitations of SVM

1. It is sensitive to noise.
2. The biggest limitation of SVM lies in the choice of the kernel.
3. Another limitation is speed and size.
4. The optimal design for multiclass SVM classifiers is also a research area.

3.7.4 Soft Margin SVM

- For the very high dimensional problems common in text classification, sometimes the data are linearly separable. But in the general case they are not, and even if they are, we might prefer a solution that better separates the bulk of the data while ignoring a few weird noise documents.
- What if the training set is not linearly separable ? *Slack variables* can be added to allow misclassification of difficult or noisy examples, resulting margin called *soft*.
- A *soft-margin* allows a few variables to cross into the margin or over the hyperplane, allowing misclassification.
- We penalize the crossover by looking at the number and distance of the misclassifications. This is a trade off between the hyperplane violations and the

margin size. The slack variables are bounded by some set cost. The farther they are from the soft margin, the less influence they have on the prediction.

- All observations have an associated slack variable,
  1. Slack variable = 0 then all points on the margin.
  2. Slack variable > 0 then a point in the margin or on the wrong side of the hyperplane.
  3. C is the tradeoff between the slack variable penalty and the margin.

3.7.5 Comparison of SVM and Neural Networks

Support vector machine	Neural network
Kernel maps to a very-high dimensional space.	Hidden layers map to lower dimensional spaces.
Search space has a unique minimum.	Search space has multiple local minima.
Very good accuracy in typical domains.	Very good accuracy in typical domains.
Kernel and cost the two parameters to select.	Requires number of hidden units and layers.
Training is extremely efficient.	Training is expensive.

Example 3.7.2 *From the following diagram, identify which data points (1, 2, 3, 4, 5) are support vectors (if any), slack variables on correct side of classifier (if any) and slack variables on wrong side of classifier (if any). Mention which point will have maximum penalty and why ?*

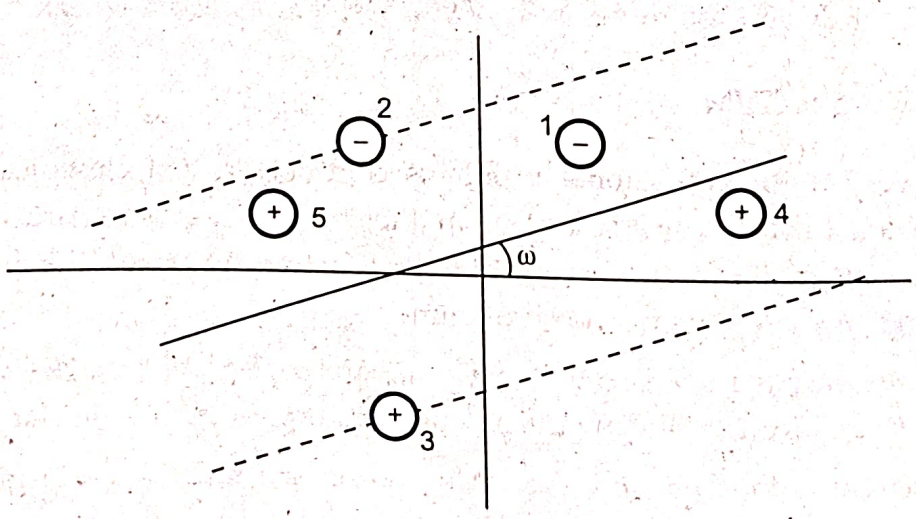


Fig. 3.7.7

Solution :

- Data points 1 and 5 will have maximum penalty.
- Margin (m) is the gap between data points & the classifier boundary. The margin is the minimum distance of any sample to the decision boundary. If this hyperplane is in the canonical form, the margin can be measured by the length of the weight vector.
- Maximal margin classifier : A classifier in the family F that maximizes the margin. Maximizing the margin is good according to intuition and PAC theory. Implies that only support vectors matter; other training examples are ignorable.
- What if the training set is not linearly separable ? Slack variables can be added to allow misclassification of difficult or noisy examples, resulting margin called soft.
- A soft-margin allows a few variables to cross into the margin or over the hyperplane, allowing misclassification.
- We penalize the crossover by looking at the number and distance of the misclassifications. This is a trade off between the hyperplane violations and the margin size. The slack variables are bounded by some set cost. The farther they are from the soft margin, the less influence they have on the prediction.
- All observations have an associated slack variable,
  1. Slack variable = 0 then all points on the margin.
  2. Slack variable > 0 then a point in the margin or on the wrong side of the hyperplane.
  3. C is the tradeoff between the slack variable penalty and the margin.

Review Questions

1. *What are support vectors and margins ? Also explain soft margin SVM.*
2. *What is slack variable ? Discuss margin errors.*
3. *Explain support vector machine.*
4. *What are support vectors and margins ? Also explain soft margin SVM.*
5. *What is slack variable ? Discuss margin errors.*
6. *Explain support vector machine.*
7. *What are the support vectors and margins ? Explain soft SVM and hard SVM.*

3.8 Kernel Functions

- Kernel methods refer to a family of widely used nonlinear algorithms for machine learning tasks like classification, regression, and feature extraction.
- Any non-linear problem (classification, regression) in the original input space can be converted into linear by making non-linear mapping ϕ into a feature space with higher dimension and shown in Fig. 3.8.1.

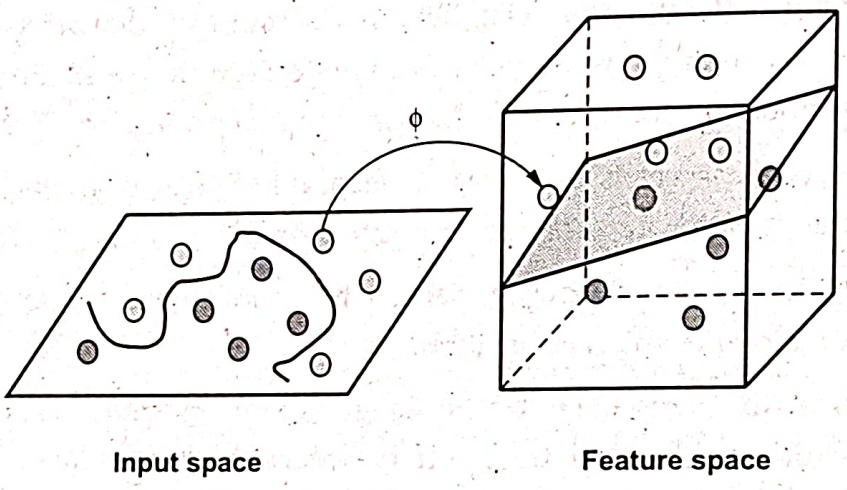


Fig. 3.8.1 Mapping of space

- Often we want to capture nonlinear patterns in the data.

a. Nonlinear Regression : Input-output relationship may not be linear

b. Nonlinear Classification : Classes may not be separable by a linear boundary

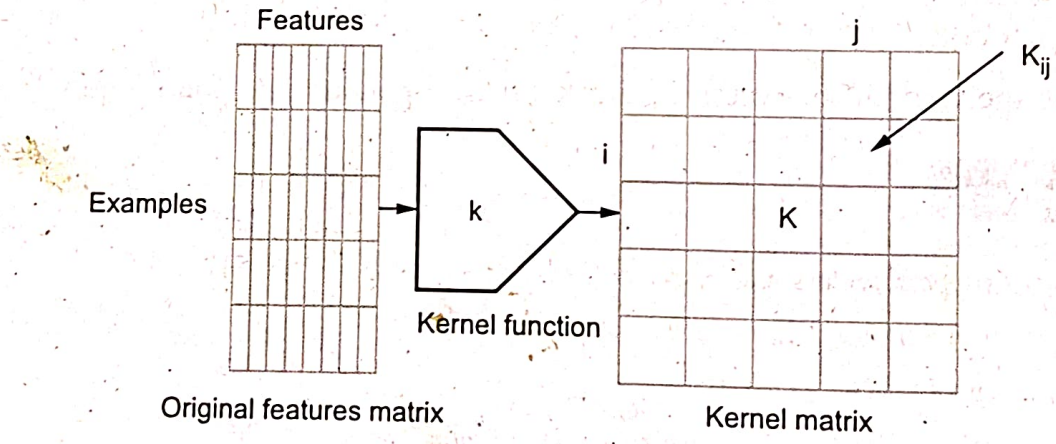


Fig. 3.8.2

- Kernels : Make linear models work in nonlinear settings.
- Kernels, using a feature mapping ϕ, map data to a new space where the original learning problem becomes easy.
- Consider two data points $x = \{x_1; x_2\}$ and $z = \{z_1; z_2\}$. Suppose we have a function k which takes as inputs x and z and computes.

$$k(x, z) = (x^T z)^2$$

$$= (x_1 z_1 + x_2 z_2)^2$$

$$= x_1^2 z_1^2 + x_2^2 z_2^2 + 2 x_1 x_2 z_1 z_2$$

$$= (x_1^2, \sqrt{2}\, x_1 x_2, x_2^2)^T (z_1^2, \sqrt{2}\, z_1 z_2\, z_2^2)$$

$$= \phi(x)^T \phi(z) \text{ (an inner product)}$$

- The above k implicitly defines a mapping ϕ to a higher dimensional space

$$\phi(x) = \{x_1^2, \sqrt{2}\, x_1 x_2 x_2^2\}$$

- We didn't need to pre-define/compute the mapping ϕ to compute k(x, z).
- The function k is known as the kernel function.
- K : N × N matrix of pairwise similarities between examples in F space.

Advantages

1. The kernel defines a similarity measure between two data points and thus allows one to incorporate prior knowledge of the problem domain.
2. Most importantly, the kernel contains all of the information about the relative positions of the inputs in the feature space and the actual learning algorithm is based only on the kernel function and can thus be carried out without explicit use of the feature space.
3. The number of operations required is not necessarily proportional to the number of features.

3.8.1 Sigmoid

- A sigmoid function produces a curve with an "S" shape. The example sigmoid function shown on the left is a special case of the logistic function, which models the growth of some set.

$$\text{sig}(t) = \frac{1}{1+e^{-t}}$$

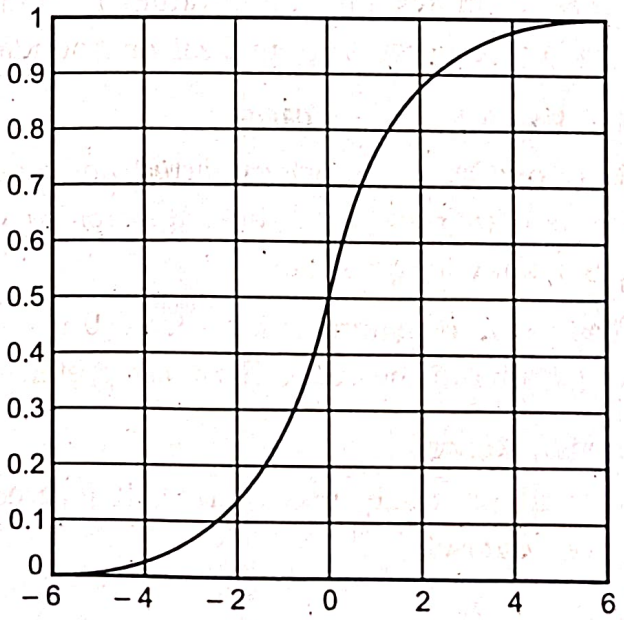


Fig. 3.8.3

- In general, a sigmoid function is real-valued and differentiable, having a non-negative or non-positive first derivative, one local minimum, and one local maximum.
- The logistic sigmoid function is related to the hyperbolic tangent as follows :

$$1 - 2\,\text{sig}(x) = 1 - 2\frac{1}{1+e^{-x}} = -\tanh\frac{x}{2}$$

- Sigmoid functions are often used in artificial neural networks to introduce nonlinearity in the model.
- A neural network element computes a linear combination of its input signals, and applies a sigmoid function to the result.
- A reason for its popularity in neural networks is because the sigmoid function satisfies a property between the derivative and itself such that it is computationally easy to perform.

$$\frac{d}{dt}\text{sig}(t) = \text{sig}(t)(1 - \text{sig}(t))$$

- Derivatives of the sigmoid function are usually employed in learning algorithms.

3.8.2 RBF, Gaussian, Polynomial

1. Polynomial Kernel

- It is a more generalized representation of the linear kernel. It is not as preferred as other kernel functions as it is less efficient and accurate.
- Polynomial Kernel Formula

$$F(x, x_i) = (x.x_j + 1)^d$$

- Here '.' shows the dot product of both the values, and d denotes the degree. $F(x, x_i)$ representing the decision boundary to separate the given classes.

2. Radial Basis Function (RBF)

- It is one of the most preferred and used kernel functions in SVM. It is usually chosen for non-linear data. It helps to make proper separation when there is no prior knowledge of data.
- The value of gamma varies from 0 to 1. You have to manually provide the value of gamma in the code. The most preferred value for gamma is 0.1.

3. Gaussian Kernel

- It is a commonly used kernel. It is used when there is no prior knowledge of a given dataset.

- Gaussian Kernel formula,

$$k(x, y) = \exp\left(\frac{||x-y||^2}{2\sigma^2}\right)$$

Review Questions

1. *Explain kernel methods for non - linearity.*
2. *What are support vector margins and also explain soft margin.*

❑❑❑

Unit IV

4 Clustering

Syllabus

IIntroduction : *What is clustering, Need of Clustering, Types of Clustering*
Hierarchical clustering algorithms /connectivity-based clustering) : *Agglomerative Hierarchical Clustering (AHC) algorithm, Divisive Hierarchical Clustering (DHC) algorithm.*
Centroid-based clustering algorithms / Partitioning clustering algorithms : *K-Means clustering algorithm, Advantages and disadvantages of K-Means clustering algorithm, Elbow method, The Silhouette method, K-Medoids, K-Prototype.*
Density-based clustering algorithms : *DBSCAN algorithm, how it works, Advantages and disadvantages of DBSCAN.*
Distribution-based clustering algorithms : *Gaussian mixture model.*
Application of Clustering Technique : *Market Segmentation, Statistical data analysis, Social network analysis, Image segmentation, Anomaly detection.*

Contents

4.1 What is Clustering ?
4.2 Hierarchical Clustering Algorithms
4.3 K-Means Clustering Algorithm
4.4 Centroid-based Clustering Algorithms
4.5 Density-based Clustering Algorithms
4.6 Distribution-based Clustering Algorithms
4.7 Application of Clustering Technique

4.1 What is Clustering ?

What is cluster analysis ?

- Given a set of objects, place them in groups such that the objects in a group are similar (or related) to one another and different from (or unrelated to) the objects in other groups.
- Cluster analysis can be a powerful data - mining tool for any organisation that needs to identify discrete groups of customers, sales transactions or other types of behaviors and things. For example, insurance providers use cluster analysis to detect fraudulent claims, and banks use it for credit scoring.
- Cluster analysis uses mathematical models to discover groups of similar customers based on the smallest variations among customers within each group.
- Cluster is a group of objects that belong to the same class. In other words the similar object are grouped in one cluster and dissimilar are grouped in other cluster.
- Clustering is a process of partitioning a set of data in a set of meaningful subclasses. Every data in the subclass shares a common trait. It helps a user understand the natural grouping or structure in a data set.
- Various types of clustering methods are partitioning methods, hierarchical clustering, Fuzzy clustering, Density - based clustering and Model - based clustering.
- Cluster analysis is process of grouping a set of data - objects into clusters.
- **Desirable properties of a clustering algorithm are as follows :**
  1. Scalability (in terms of both time and space).
  2. Ability to deal with different data types.
  3. Minimal requirements for domain knowledge to determine input parameters.
  4. Interpretability and usability.
- Clustering of data is a method by which large sets of data are grouped into clusters of smaller sets of similar data. Clustering can be considered the most important unsupervised learning problem.
- A cluster is therefore a collection of objects which are "similar" between them and are "dissimilar" to the objects belonging to other clusters. Fig. 4.1.1 shows cluster.

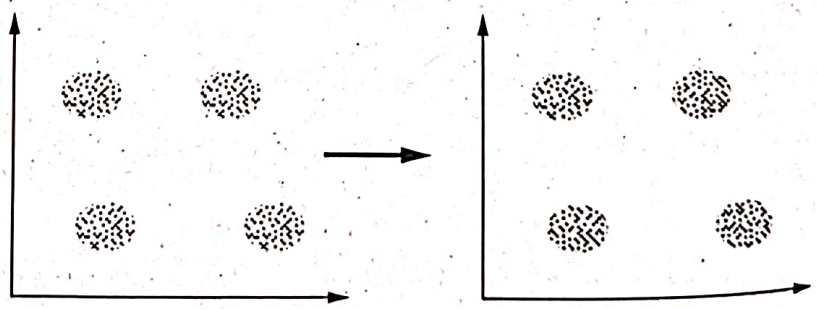
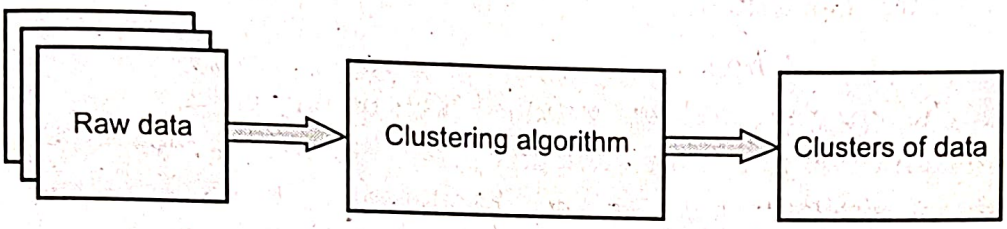
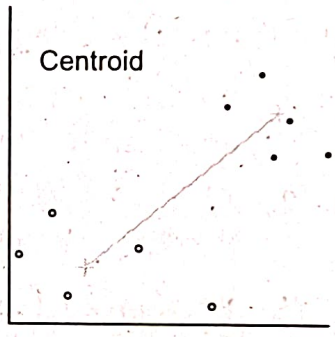


Fig. 4.1.1 Cluster

- In this case we easily identify the 4 clusters into which the data can be divided ; the similarity criterion is distance : Two or more objects belong to the same cluster if they are "close" according to a given distance (in this case geometrical distance). This is called **distance-based clustering.**



- Clustering means grouping of data or dividing a large data set into smaller data sets of some similarity.
- A clustering algorithm attempts to find natural groups of components or data based on some similarity. Also, the clustering algorithm finds the centroid of a group of data sets.



- To determine cluster membership, most algorithms evaluate the distance between a point and the cluster centroids. The output from a clustering algorithm is basically a statistical description of the cluster centroids with the number of components in each cluster.
- **Cluster centroid :** The centroid of a cluster is a point whose parameter values are the mean of the parameter values of all the points in the clusters. Each cluster has a well defined centroid.
- **Distance :** The distance between two points is taken as a common metric to as see the similarity among the components of a population. The commonly used distance measure is the euclidean metric which defines the distance between two points $p = (p_1, p_2, ...)$ and $q = (q_1, q_2, ...)$ is given by :

$$d = \sum_{i=1}^{k} (p_i - q_i)^2$$

- The goal of clustering is to determine the intrinsic grouping in a set of unlabeled data. But how to decide what constitutes a good clustering ? It can be shown that there is no absolute "best" criterion which would be independent of the final aim of the clustering. Consequently, it is the user which must supply this criterion, in such a way that the result of the clustering will suit their needs.
- Clustering analysis helps construct meaningful partitioning of a large set of objects. Cluster analysis has been widely used in numerous applications, including pattern recognition, data analysis, image processing, etc.

- Clustering algorithms may be classified as listed below :
  1. Exclusive clustering.
  2. Overlapping clustering.
  3. Hierarchical clustering.
  4. Probabilistic clustering.
- A good clustering method will produce high quality clusters with high intra-class similarity and low inter-class similarity. The quality of a clustering result depends on both the similarity measure used by the method and its implementation. The quality of a clustering method is also measured by its ability to discover some or all of the hidden patterns.

Examples of Clustering Applications

1. **Marketing :** Help marketers discover distinct groups in their customer bases and then use this knowledge to develop targeted marketing programs.
2. **Land use :** Identification of areas of similar land use in an earth observation database.
3. **Insurance :** Identifying groups of motor insurance policy holders with a high average claim cost.
4. **Urban planning :** Identifying groups of houses according to their house type, value, and geographical location.
5. **Seismology :** Observed earth quake epicenters should be clustered along continent faults.

4.1.1 Typical Requirements of Clustering in Data Mining

1. **Scalability :** Many clustering algorithms work well on small data sets.
2. **Ability to deal with different types of attributes :** Many algorithms are designed to cluster interval-based data.
3. **Discovery of clusters with arbitrary shape :** Many clustering algorithms determine clusters based on Euclidean or Manhattan distance measures.
4. **Minimal requirements for domain knowledge to determine input parameters :** Many clustering algorithms require users to input certain parameters in cluster analysis.
5. **Ability to deal with noisy data :** Most real - world databases contain outliers or missing, unknown or erroneous data. Some clustering algorithms are sensitive to such data and may lead to clusters of poor quality.
6. **Incremental clustering and insensitivity to the order of input records :** Some clustering algorithms cannot incorporate newly inserted data into existing clustering structures.

7. **High dimensionality :** A database or a data warehouse can contain several dimensions or attributes.
8. **Constraint-based clustering :** Real-world applications may need to perform clustering under various kinds of constraints.
9. **Interpretability and usability :** Users expect clustering results to be interpretable, comprehensible and usable.

4.1.2 Problems with Clustering

1. Current clustering techniques do not address all the requirements adequately ;
2. Dealing with large number of dimensions and large number of data items can be problematic because of time complexity ;.
3. The effectiveness of the method depends on the definition of "distance" ;
4. If an obvious distance measure doesn't exist we must "define" it, which is not always easy, especially in multi-dimensional spaces ;
5. The result of the clustering algorithm can be interpreted in different ways.

4.1.3 Types of Clusters

- Type of clusters are as follows :
  a) Well - separated clusters
  b) Prototype - based clusters
  c) Contiguity - based clusters
  d) Density - based clusters

a) Well - separated clusters :

- A cluster is a set of points such that any point in a cluster is closer to every other point in the cluster than to any point not in the cluster.
- Fig. 4.1.2 shows well-separated cluster.

Fig. 4.1.2 Well-separated cluster

- Sometimes a threshold is used to specify that all the objects in a cluster must sufficiently close to one another. Definition of a cluster is satisfied only when the data contains natural clusters.

b) Prototype - based cluster :

- A cluster is a set of objects such that an object in a cluster is closer (more similar) to the prototype or "center" of a cluster, than to the center of any other cluster. Prototype based clusters can also be referred to as "Center-Based" Clusters.
- The center of a cluster is often a centroid, the average of all the points in the cluster, or a medoid, the most "representative" point of a cluster. Fig. 4.1.3 shows 4 center-based clusters.

Fig. 4.1.3 4 Center-based clusters

- If the data is numerical, the prototype of the cluster is often a centroid i.e., the average of all the points in the cluster.
- If the data has categorical attributes, the prototype of the cluster is often a medoid i.e., the most representative point of the cluster.
- Objects in the cluster are closer to the prototype of the cluster than to the prototype of any other cluster.
- K-Means and K-Medoids are the examples of prototype-based clustering algorithm.

c) Contiguity - based clusters :

- A cluster is a set of points such that a point in a cluster is closer (or more similar) to one or more other points in the cluster than to any point not in the cluster.

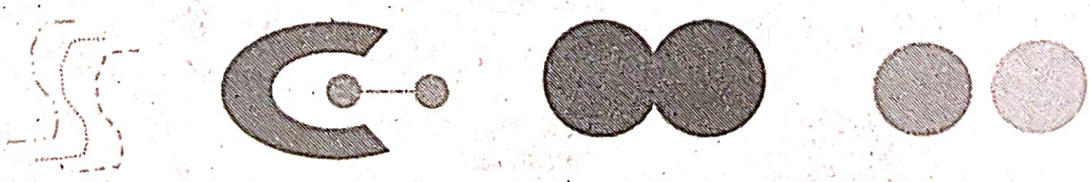
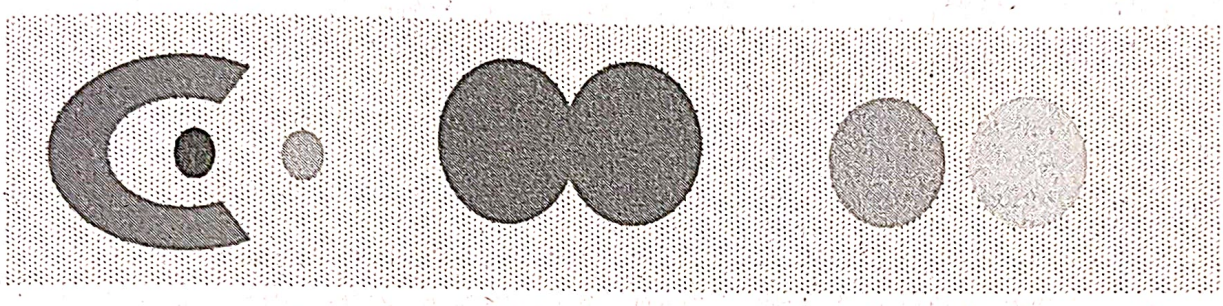


Fig. 4.1.4 8 contiguous clusters

d) Density - based :

- A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density.
- Used when the clusters are irregular or intertwined and when noise and outliers are present.

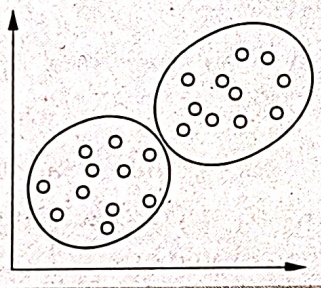
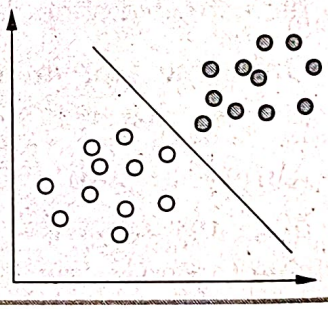


4.1.4 Desired Features of Cluster Analysis

- Features are as follows :

1. **Scalability :** Data-mining problems can be large and therefore a cluster-analysis method should be able to deal with large problems gracefully. Ideally, performance should be linear with data-size.
2. **Only one scan of the dataset :** For large problems, data must be stored on disk, so cost of I/O disk can become significant in solving the problem.
3. **Ability to stop and resume :** For large dataset, cluster-analysis may require huge processor-time to complete the task. In such cases, the task should be able to be stopped and then resumed when convenient.
4. **Minimal input parameters :** The method should not expect too much guidance from the data-mining analyst.
5. **Robustness :** Most data obtained from a variety of sources has errors. Therefore, the method should be able to deal with noise, outlier and missing values gracefully.
6. **Ability to discover different cluster-shapes :** Clusters appear in different shapes and not all clusters are spherical. So the method should be able to discover cluster-shapes other than spherical.
7. **Different data types :** Many problems have a mixture of data types, for e.g. numerical, categorical and even textual. Therefore, the method should be able to deal with numerical, boolean and categorical data.
8. **Result independent of data input order :** The method should not be sensitive to data input-order.

4.1.5 Difference between Clustering and Classification

Sr. No.	Clustering	Classification
1.	This function maps the data into one of several clusters which is the grouping of data items based on the similarities between them.	This model function classifies the data into one of several predefined categorical classes.
2.	Involved in unsupervised learning	Involved in supervised learning
3.	Training sample is not provided.	Training sample is provided,
4.	The number of cluster is not known before clustering, These are identified after the completion of clustering.	The number of classes is known before classification as there is predefined output based on input data.
5.	Data is not labeled.	Labeled data points.
6.	Asks how can I group this set of items ?	Asks what class does this item belong to ?
7.	Unknown number of classes.	Known number of classes.
8.	Used to understand data.	Used to classify future observations.
9.		

4.2 Hierarchical Clustering Algorithms

- Hierarchical clustering arranges items in a hierarchy with a tree like structure based on the distance or similarity between them. The graphical representation of the resulting hierarchy is a tree-structured graph called a **dendrogram.**
- The tree is not a single set of clusters, but rather a multilevel hierarchy, where clusters at one level are joined as clusters at the next level.
- The hierarchical clustering algorithm is an unsupervised machine learning technique.
- Hierarchical clustering starts with k = N clusters and proceed by merging the two closest objects into one cluster, obtaining k = N – 1 clusters. The process of merging two clusters to obtain k – 1 clusters is repeated until we reach the desired number of clusters k.

- Fig. 4.2.1 shows type of hierarchical clustering. Divisive and Agglomerative are two types of hierarchical clustering.

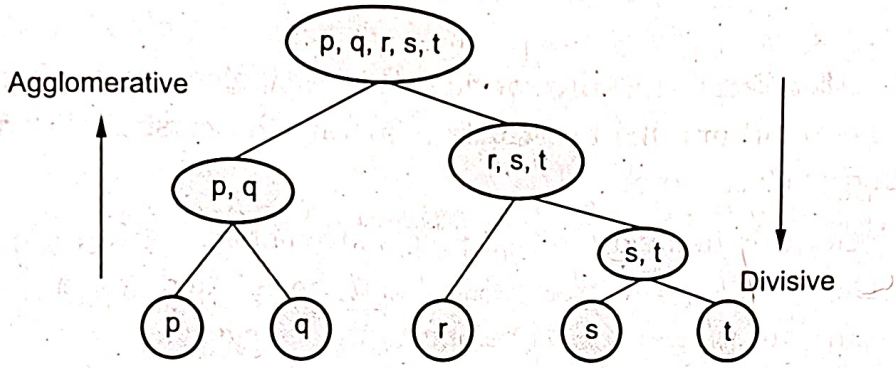


Fig. 4.2.1 Types of hierarchical clustering

4.2.1 Advantages and Disadvantages of Hierarchical Clustering

1. Advantages

- It is simple to implement.
- It is easy and results in a hierarchy, a structure that contains more information.
- It does not need us to pre-specify the number of clusters.

2. Disadvantages

- It breaks the large clusters.
- It is Difficult to handle different sized clusters and convex shapes.
- It is sensitive to noise and outliers.
- The algorithm can never be changed or deleted once it was done previously.

4.2.2 Divisive Methods

- Divisive clustering is known as the top - down approach. Divisive methods initialize with all examples as members of a single cluster and split this cluster recursively.
- Start at the top with all documents in one cluster. The cluster is split using a flat clustering algorithm. This procedure is applied recursively until each document is in its own singleton cluster.
- It subdivides the clusters into smaller and smaller pieces, until each object form a cluster on its own or until it satisfies certain termination conditions, such as a desired number of cluster or the diameter of each cluster is within a certain threshold.
- The divisive hierarchical clustering, also known as DIANA (DIvisive ANAlysis) is the inverse of agglomerative clustering.
- Divisive clustering is good at identifying large clusters.

4.2.3 Agglomerative Clustering

- It is also known as AGNES (Agglomerative Nesting). It works in a bottom-up manner.
- That is, each object is initially considered as a single-element cluster (leaf). At each step of the algorithm, the two clusters that are the most similar are combined into a new bigger cluster (nodes).
- This procedure is iterated until all points are member of just one single big cluster (root). The result is a tree which can be plotted as a dendrogram. Most hierarchical clustering methods belong to this category.
- Initially, AGNES places each objects into a cluster of its own. The clusters are then merged step-by-step according to some criterion.
- For example, cluster C_1 and C_2 may be merged if an object in C_1 and object in C_2 form the minimum Euclidean distance between any two objects from different clusters.
- In the agglomerative hierarchical approach, we start by defining each data point to be a cluster and combine existing clusters at each step. Here are four different methods for doing this :
  1. **Single linkage :** Smallest pairwise distance between elements from each cluster. It tends to produce long, "loose" clusters.
  2. **Complete linkage :** Largest distance between elements from each cluster. It tends to produce more compact clusters.
  3. **Average linkage :** The average distance between elements from each cluster. It can vary in the compactness of the clusters it creates.
  4. **Centroid linkage :** Distance between cluster means.
- Agglomerative clustering is good at identifying small clusters.
- Steps for an agglomerative hierarchical cluster analysis.
  1. Find the similarity or dissimilarity between every pair of objects in the data set. In this step, we calculate the distance between objects using the pdist function. The pdist function supports many different ways to compute this measurement.
  2. Group the objects into a binary, hierarchical cluster tree. In this step, we link pairs of objects that are in close proximity using the linkage function. The linkage function uses the distance information generated in step 1 to determine the proximity of objects to each other. As objects are paired into binary clusters, the newly formed clusters are grouped into larger clusters until a hierarchical tree is formed.

3. Determine where to cut the hierarchical tree into clusters. In this step, we use the cluster function to prune branches off the bottom of the hierarchical tree, and assign all the objects below each cut to a single cluster. This creates a partition of the data. The cluster function can create these clusters by detecting natural groupings in the hierarchical tree or by cutting off the hierarchical tree at an arbitrary point.

4.2.4 Difference between Agglomerative and Divisive Clustering

Sr. No.	Agglomerative clustering	Divisive clustering
1.	Agglomerative clustering is known as bottom-up approach.	Divisive clustering is known as the top-down approach.
2.	Agglomerative clustering is good at identifying small clusters.	Divisive clustering is good at identifying large clusters.
3.	Each cluster starts with only one object.	Start with all object in one cluster.
4.	It is also known as AGNES (Agglomerative Nesting).	It is also known as DIANA (Divise Analysis).
5.	Iteratively clusters are merged together.	Large clusters are successively divided.

Example 4.2.1 *Consider one dimensional data set {7, 10, 20, 28, 35}, perform hierarchical clustering and plot the dendrogram to visualize it.*

Solution : Draw the graph using data set.

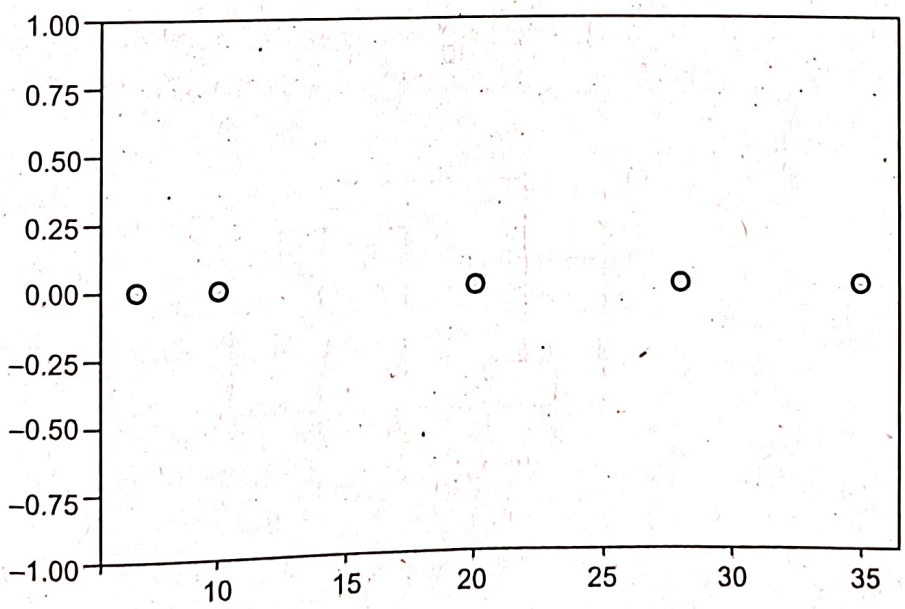


Fig. 4.2.2

- From the above graph, we write :
  1. Cluster 1 : The first two points (7 and 10) are close to each other and should be in the same cluster.
  2. Cluster 2 : The last two points (28 and 35) are close to each other and should be in the same cluster.
  3. Cluster of the center point (20) is not easy to conclude.
- Solve the problem by hand using both the types of agglomerative hierarchical clustering :

a) Single linkage :

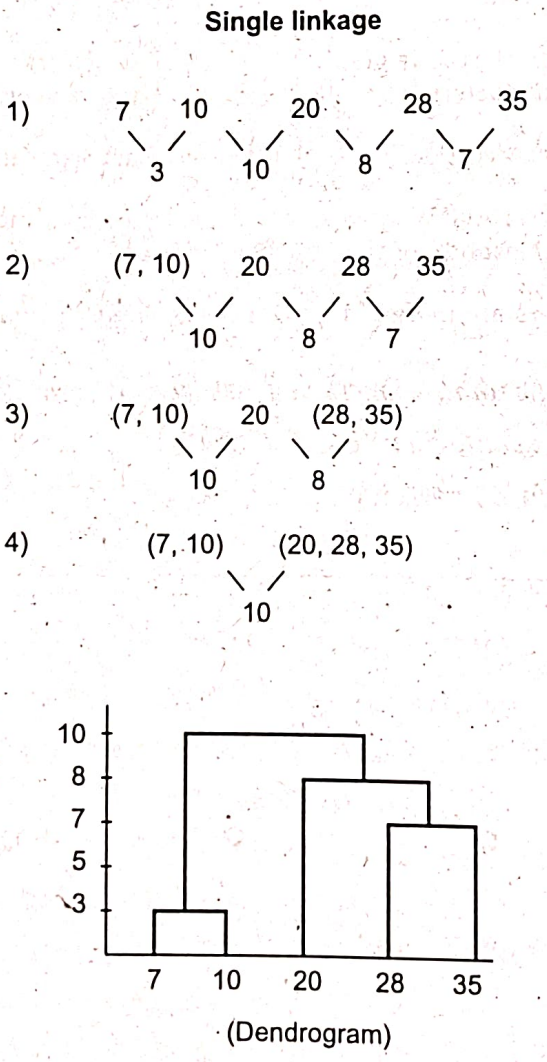


Fig. 4.2.3

- Using single linkage two clusters are formed :
  Cluster 1 : (7, 10), Cluster 2 : (20, 28, 35)

b) Complete linkage :

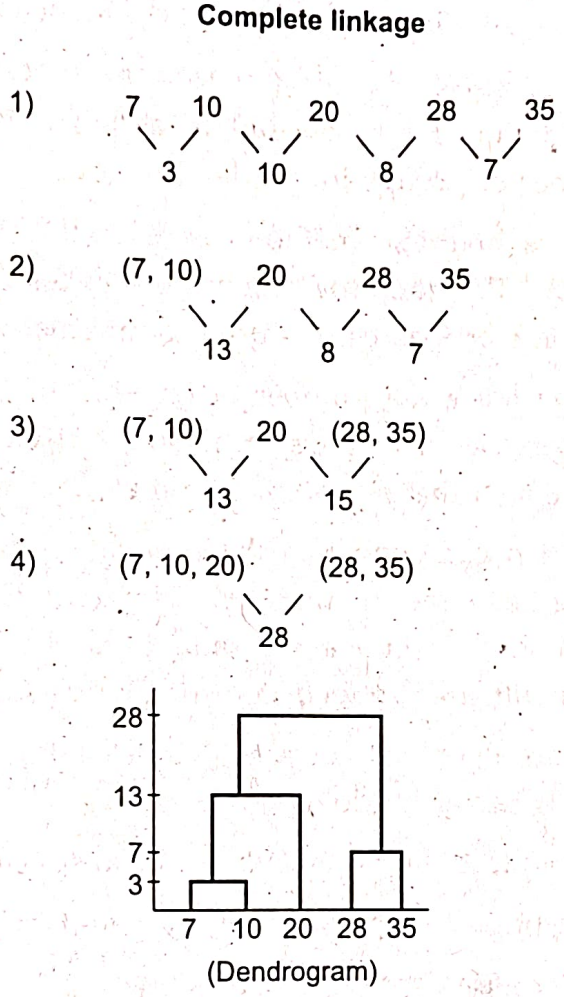


Fig. 4.2.4

- Using complete linkage two clusters are formed : Cluster 1 : (7, 10, 20), Cluster 2 : (28, 35)

Review Question

1. *What is agglomerative clustering ? Explain with example.*

4.3 K-Means Clustering Algorithm

- In K-means clustering, the objects are divided into several clusters mentioned by the number 'K.' So if we say K = 2, the objects are divided into two clusters, c_1 and c_2, as shown in Fig. 4.3.1.
- K-means clustering is an unsupervised learning algorithm. There is no labeled data for this clustering, unlike in supervised

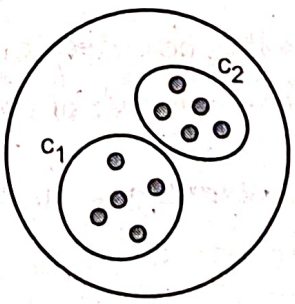


Fig. 4.3.1 Two clusters

learning. K-means performs the division of objects into clusters that share similarities and are dissimilar to the objects belonging to another cluster.

- K-means is a centroid-based clustering algorithm, where we calculate the distance between each data point and a centroid to assign it to a cluster. The goal is to identify the K number of groups in the dataset.
- K-means clustering is heuristic method. Here each cluster is represented by the center of the cluster. "K" stands for number of clusters, it is typically a user input to the algorithm ; some criteria can be used to automatically estimate K.
- This method initially takes the number of components of the population equal to the final required number of clusters. In this step itself the final required number of clusters is chosen such that the points are mutually farthest apart.
- Next, it examines each component in the population and assigns it to one of the clusters depending on the minimum distance. The centroid's position is recalculated everytime a component is added to the cluster and this continues until all the components are grouped into the final required number of clusters.
- Given K, the K-means algorithm consists of four steps :
  1. Select initial centroids at random.
  2. Assign each object to the cluster with the nearest centroid.
  3. Compute each centroid as the mean of the objects assigned to it.
  4. Repeat previous 2 steps until no change.
- The $x_1, ..., x_N$ are data points or vectors of observations. Each observation (vector x_i) will be assigned to one and only one cluster. The C(i) denotes cluster number for the observation. K-means minimizes within-cluster point scatter :

$$W(C) = \frac{1}{2} \sum_{K-1}^{K} \sum_{C(i)-K} \sum_{C(i)-K} ||x_i - x_j||^2$$

$$= \sum_{K-1}^{K} NK \sum_{C(i)-K} ||x_i - m_K||^2$$

where,

m_K is the mean vector of the K^{th} cluster.

N_K is the number of observations in K^{th} cluster.

K-means algorithm properties

1. There are always K clusters.
2. There is always at least one item in each cluster.

3. The clusters are non-hierarchical and they do not overlap.
4. Every member of a cluster is closer to its cluster than any other cluster because closeness does not always involve the 'center' of clusters.

The K-means algorithm process

1. The dataset is partitioned into K clusters and the data points are randomly assigned to the clusters resulting in clusters that have roughly the same number of data points.
2. For each data point,
   a. Calculate the distance from the data point to each cluster.
   b. If the data point is closest to its own cluster, leave it where it is.
   c. If the data point is not closest to its own cluster, move it into the closest cluster.
3. Repeat the above step until a complete pass through all the data points results in no data point moving from one cluster to another. At this point the clusters are stable and the clustering process ends.
4. The choice of initial partition can greatly affect the final clusters that result, in terms of inter-cluster and intracluster distances and cohesion.

- K-means algorithm is iterative in nature. It converges, however only a local minimum is obtained. It works only for numerical data. This method easy to implement.

Advantages of K-means algorithm :

1. Efficient in computation.
2. Easy to implement.

Weaknesses :

1. Applicable only when mean is defined.
2. Need to specify K, the number of clusters, in advance.
3. Trouble with noisy data and outliers.
4. Not suitable to discover clusters with non-convex shapes.

4.3.1 K-medoids

- The K-medoids algorithm is a clustering algorithm related to the K-means algorithm and the medoidshift algorithm. K-medoid is a classical partitioning technique of clustering that clusters the data set of n objects into K clusters known a priori. A useful tool for determining K is the silhouette.

- The most common realisation of K-medoid clustering is the Partitioning Around Medoids (PAM) algorithm. PAM uses a greedy search which may not find the optimum solution, but it is faster than exhaustive search.
- Instead of taking the mean value of the object in a cluster as a reference point, medoids can be used, which is the most centrally located object in a cluster.

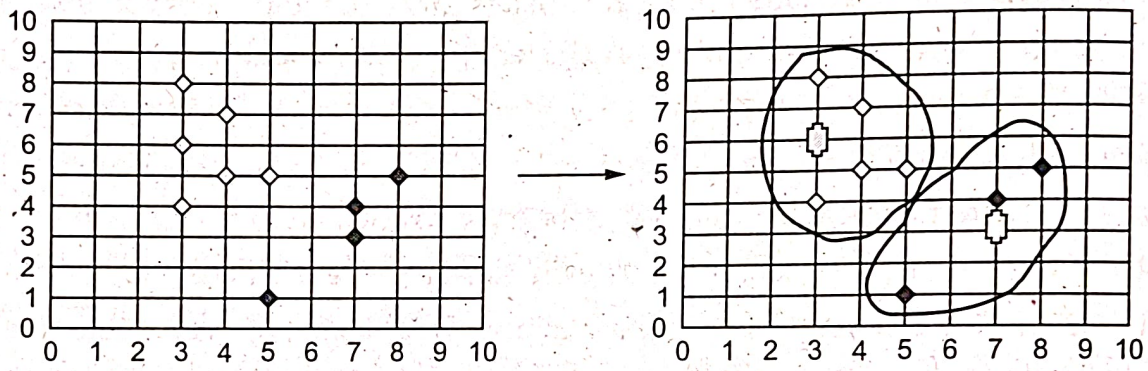


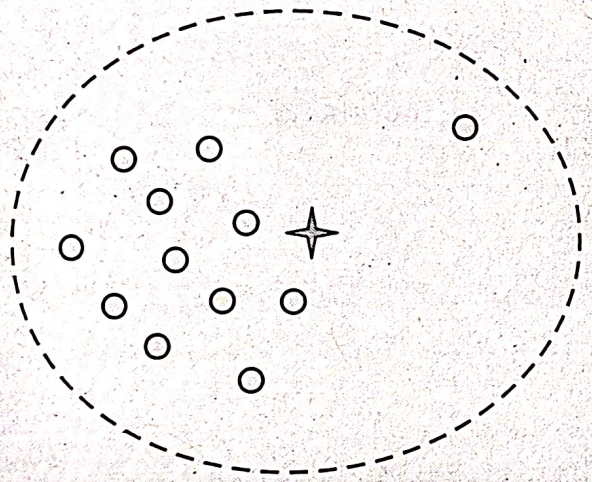
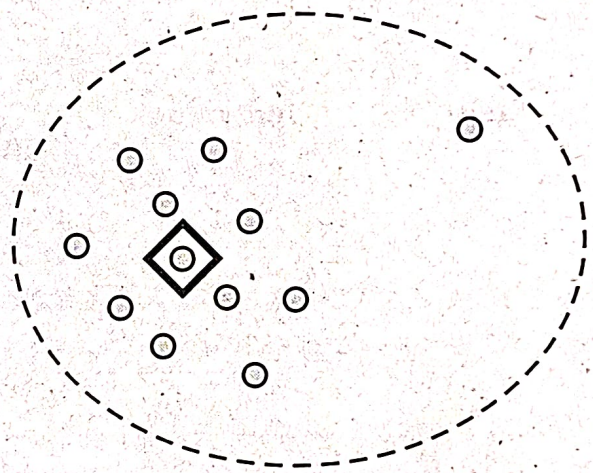
Fig. 4.3.2

- A medoid can be defined as that object of a cluster, whose average dissimilarity to all the objects in the cluster is minimal i.e. it is a most centrally located point in the given data set.

1. The algorithm begins with arbitrary selection of the K objects as medoid points out of n data points (n > K).
2. After selection of the K medoid points, associate each data object in the given data set to most similar medoid. The similarity here is defined using distance measure that can be euclidean distance, manhattan distance or minkowski distance.
3. Randomly select nonmedoid object O'.
4. Compute total cost, S of swapping initial medoid object to O'.
5. If S < 0, then swap initial medoid with the new one (if S < 0 then there will be new set of medoids).
6. Repeat steps 2 to 5 until there is no change in the medoid.

4.3.2 Difference between K-mean and K-medoids Clustering

K-means	K-medoids
Each cluster is represented by the center of the cluster.	Each cluster is represented by one of the objects in the cluster.
Simple Centroid-based method.	Data point are chosen by the medoids.

K-means clustering is a non-hierarchical cluster analysis method that attempts to partition existing.	K-medoid is a classic partition clustering technique that groups data sets of ni objects into K groups.
objects into one or more clusters or groups of objects based on their characteristics.	Known a priori.
K-means algorithm is very prone to the effects of outliers.	Normally less delicate to outliers than K-means.
Convex shape is required.	Convex shape is not required.
K-means clustering algorithm is sensitive to outliers.	K-medoids clustering is more robust to noises and outliers.
	

Example 4.3.1 *Consider the following data set consisting of the scores of two variables on each of seven individuals : Apply K-mean clustering to cluster this data into 2 clusters.*

Subject	*A*	*B*
1	*1.0*	*1.0*
2	*1.5*	*2.0*
3	*3.0*	*4.0*
4	*5.0*	*7.0*
5	*3.5*	*5.0*
6	*4.5*	*5*
7	*3.5*	*4.5*

Solution : Given data set is grouped into two clusters. First find the initial partition, so the A & B values of the two individuals furthest apart (using Euclidean distance), define the initial cluster :

	Individual	Mean vector (Centroid)
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

- In sequence, remaining individuals are examined and allocated to the cluster to which they are closest, using Euclidean distance to the cluster mean. Each time, mean vector is recalculated and new member is added. Following are the steps :

	Cluster 1		Cluster 2	
Step	Individual	Mean Vector (Centroid)	Individual	Mean Vector (Centroid)
1	1	(1.0, 1.0)	4	(5.0, 7.0)
2	1, 2	(1.2, 1.5)	4	(5.0, 7.0)
3	1, 2, 3	(1.8, 2.3)	4	(5.0, 7.0)
4	1, 2, 3	(1.8, 2.3)	4, 5	(4.2, 6.0)
5	1, 2, 3	(1.8, 2.3)	4, 5, 6	(4.3, 5.7)
6	1, 2, 3	(1.8, 2.3)	4, 5, 6, 7	(4.1, 5.4)

- Now the initial partition has changed and the two clusters at this stage having the following characteristics :

	Individual	Mean vector (Centroid)
Cluster 1	1, 2, 3	(1.8, 2.3)
Cluster 2	4, 5, 6, 7	(4.1, 5.4)

- Compare each individuals distance to its own cluster mean and to that of the opposite cluster because individual cluster assign to right cluster is not confirmed.

Individual	Distance to mean (Centroid) of cluster 1	Distance to mean (Centroid) of cluster 2
1	1.5	5.4
2	0.4	4.3
3	2.1	1.8

4	5.7	1.8
5	3.2	0.7
6	3.8	0.6
7	2.8	1.1

- Each individual's distance to its own cluster mean should be smaller than the distance to the other cluster's mean (which is not the case with individual 3). Thus, individual 3 is relocated to cluster 2 resulting in the new partition :

	Individual	Mean vector (Centroid)
Cluster 1	1, 2	(1.3, 1.5)
Cluster 2	3, 4, 5, 6, 7	(3.9, 5.1)

Review Question

1. *Justify with elaboration the following statement : The K-means algorithm is based on the strong initial condition to decide the number of clusters through the assignment of 'K' initial centroids or means.*

4.4 Centroid-based Clustering Algorithms

- Centroid based clustering is often used in game analytics, primarily due to popularity and widespread use of k-means clustering.
- Centroid based models represent clusters by a central vector, which does not need to be an actual object. The analyst needs to define the number of clusters in advance and k-means clustering then targets the optimization problem of finding k centers and assigning objects to the nearest center, in a way that minimizes the squared distances.
- K-means finds local optima, not global optima and is therefore typically run multiple times with randomized initializations. Modifications of the k-means algorithm such as k-medoids and spherical k-means allow for using other distance measures than Euclidean distance.

4.4.1 Elbow Method

- The elbow method is used to determine the optimal number of clusters in k-means clustering. The elbow method plots the value of the cost function produced by different values of k.

- If k increases, average distortion will decrease, each cluster will have fewer constituent instances and the instances will be closer to their respective centroids. However, the improvements in average distortion will decline as k increases. The value of k at which improvement in distortion declines the most is called the elbow, at which we should stop dividing the data into further clusters.
- It involves running the algorithm multiple times over a loop, with an increasing number of cluster choice and then plotting a clustering score as a function of the number of clusters.
- The Elbow and Silhouette methods are the two state-of-the-art methods used to identify the correct cluster number in the dataset.
- The Elbow method is the oldest method to distinguish the potential optimal cluster number for the analyzed dataset, whose basic idea is to specify K = 2 as the initial optimal cluster number K and then keeps increasing K by step 1 to the maximal specified for the estimated potential optimal cluster number, and finally distinguish the potential optimal cluster number K corresponding to the plateau.
- The optimal cluster number K is distinguished by the fact that before reaching K, the cost rapidly decreases to the called cost peak value, and after exceeding K, it continues to increase with the called cost peak value almost unchanged, as shown in Fig. 4.4.1 (a) with an explicit elbow point.

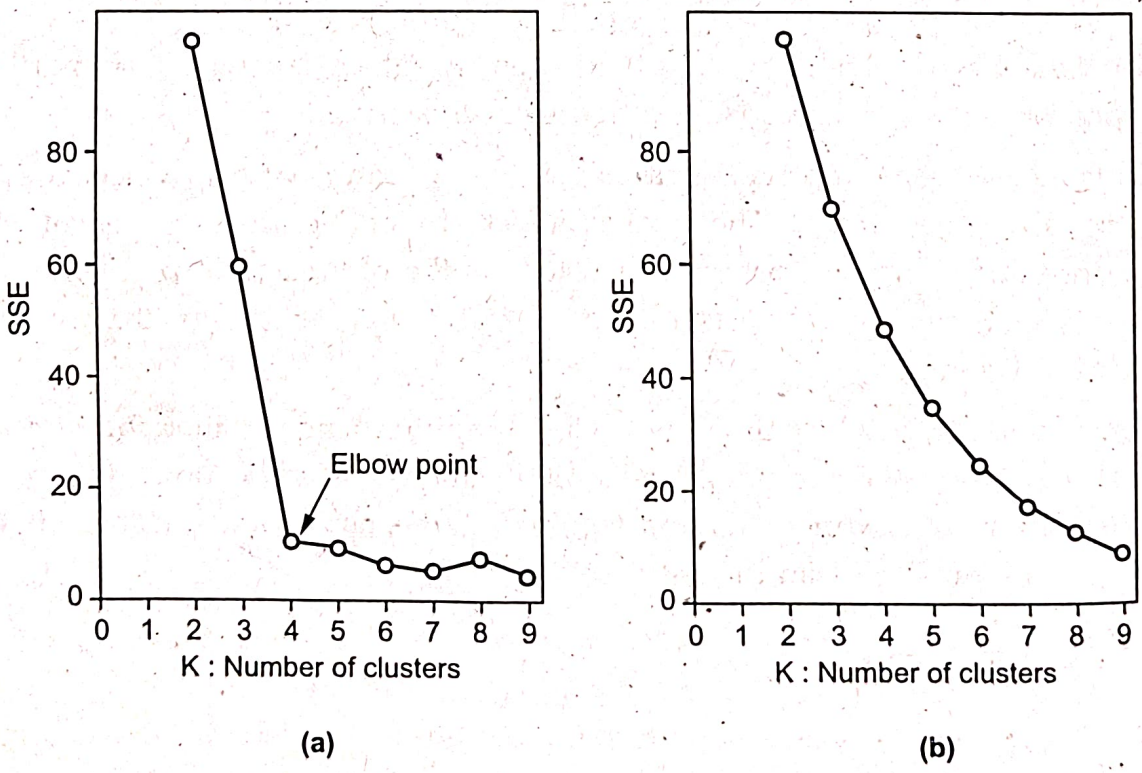


Fig. 4.4.1 Elbow point

- Meanwhile, the optimal cluster number corresponding to the elbow point depends on the manmade selection. There is, however, a problem with the Elbow method in that the elbow point cannot be unambiguously distinguished by the experienced analysts when the plotted curve is fairly smooth, as shown in Fig. 4.4.1 (b) with an ambiguous elbow point.
- To select the best K, we need to plot the mean in-cluster distance for each K. As K increases from 1, before reaching the optimal K, the decrease speed is relatively fast because the number of centers are too low from the very beginning and each new center will incur a large decrease in the mean distance.
- But after the optimal K, the decrease is slow since the correct cluster structure is already discovered and any newly added center will appear in a certain cluster already formed. That will not decrease the mean in-cluster distance too much. The entire curve looks like an L shape and the best K lies in the turning point or the elbow of the L shape.

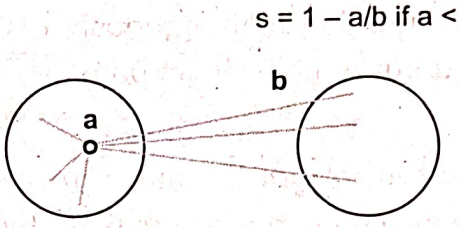
4.4.2 The Silhouette Method

- Silhouette refers to a method of interpretation and validation of clusters of data.
- Silhouettes are a general graphical aid for interpretation and validation of cluster analysis. This technique is available through the silhouette function. In order to calculate silhouettes, two types of data are needed :
  1. The collection of all distances between objects. These distances are obtained from application of dist function on the coordinates of the elements in mat with argument method.
  2. The partition obtained by the application of a clustering technique.
- For each element, a silhouette value is calculated and evaluates the degree of confidence in the assignment of the element :
  1. Well-clustered elements have a score near 1.
  2. Poorly-clustered elements have a score near – 1.
- Thus, silhouettes indicate the objects that are well or poorly clustered. Silhouette coefficient combines ideas of both cohesion and separation, but for individual points, as well as clusters and clustering's.
- For an individual point, I

  a = Average distance of i to the points in the same cluster.

  b = Min (average distance of i to points in another cluster).

Silhouette coefficient of i :



Silhouette coefficient

- **Cohesion :** Measures how closely related are objects in a cluster.
- **Separation :** Measure how distinct or well-separated a cluster is from other clusters.

4.5 Density-based Clustering Algorithms

- Density-based clustering is unsupervised learning methodologies used in model building and machine learning algorithms. This is a clustering method that is used in machine learning to separate clusters of high density from clusters of low density.
- DBSCAN stands for "Density-Based Spatial Clustering of Applications with Noise." DBSCAN groups together closely-packed points.
- There are two inputs to DBSCAN :
  1. The search distance around point (∈).
  2. The minimum number of points (minpts) required to form a density cluster.
- DBSCAN is based on this intuitive notion of "clusters" and "noise". The key idea is that for each point of a cluster, the neighborhood of a given radius has to contain at least a minimum number of points.
- The parameter (∈) defines the radius of neighborhood around a point x. It is called the ∈-neighborhood of x. The parameter MinPts is the minimum number of neighbors within "eps" radius.
- Any point x in the dataset, with a neighbor count greater than or equal to MinPts, is marked as a core point. We say that x is border point, if the number of its neighbors is less than MinPts, but it belongs to the ∈-neighborhood of some core point z. Finally, if a point is neither a core nor a border point, then it is called a noise point or an outlier.

- Fig. 4.5.1 shows DBSCAN.

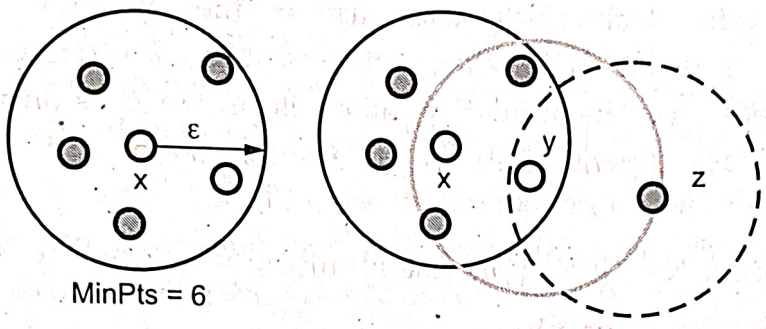


Fig. 4.5.1 DBSCAN

- We define following terms for understanding the DBSCAN algorithm :
  1. **Direct density reachable :** A point "A" is directly density reachable from another point "B" if : "A" is in the ∈-neighborhood of "B" and "B" is a core point.
  2. **Density reachable :** A point "A" is density reachable from "B" if there are a set of core points leading from "B" to "A.
  3. **Density connected :** Two points "A" and "B" are density connected if there are a core point "C", such that both "A" and "B" are density reachable from "C".
- DBSCAN is broadly used in many applications such as market research, pattern recognition, data analysis and image processing.

4.5.1 Advantages and Disadvantages of DBSCAN

Advantages :

a) We don't need to specify the number of clusters.

b) Flexibility in the shapes & sizes of clusters.

c) Able to deal with noise and outliers.

d) Ability to identify uneven shapes.

e) It is easy for someone who knows the dataset, to set the parameters.

Disadvantages :

1. Input parameters may be difficult to determine.
2. In some situations very sensitive to input parameter setting.
3. Can be confused when there is a border point that belongs to two clusters.
4. Results depend highly on the distance metric.
5. Can be hard to guess the correct parameters for an unknown dataset.

4.5.2 Spectral Clustering

- Spectral clustering techniques make use of the spectrum (eigenvalues) of the similarity matrix of the data to perform dimensionality reduction before clustering in fewer dimensions. The similarity matrix is provided as an input and consists of a quantitative assessment of the relative similarity of each pair of points in the dataset.
- Given data points $x_1, \ldots x_N$, pairwise affinities $A_{ij} = A(x_i, x_j)$.
- Build similarity graph shown in Fig. 4.5.2.
- Clustering = Find a cut through the graph.
- Define a cut-type objective function.
- The low-dimensional space is determined by the data. Spectral clustering makes use of the spectrum of the graph for dimensionality reduction.

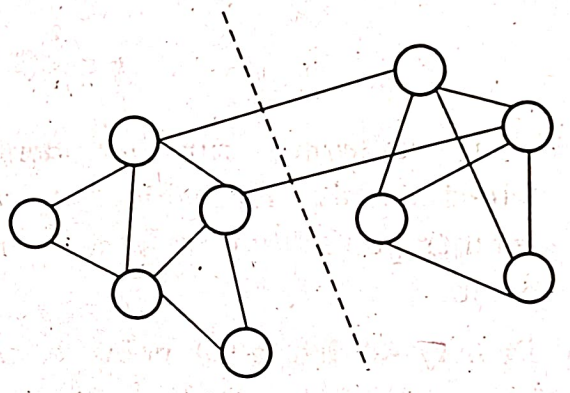


Fig. 4.5.2 Similarity graph

- Projection and clustering equates to graph partition by different min-cut criteria.

Advantages :

1. Does not make strong assumptions on the statistics of the clusters.
2. Easy to implement.
3. Good clustering results.
4. Reasonably fast for sparse data sets of several thousand elements.

Disadvantages :

1. May be sensitive to choice of parameters.
2. Computationaly expensive for large datasets.

4.6 Distribution-based Clustering Algorithms

- Distribution-based clustering directly relates to the use of distribution models (e.g. Gaussian/Normal) in statistics. Distribution-based clustering starts with the assumption that each cluster is made up of points sampled from a known distribution, usually the normal distribution. The goal of the algorithm is to find the parameters of the distribution of each cluster.

- Fundamentally, clusters are defined based on how likely the objects included are likely to belong to the same distribution.
- Distribution-based models can provide information beyond the cluster assignments of objects, for example correlation of object attributes, but suffer from overfitting problems if the complexity of the model used is not constrained.
- These models do not work if there is no mathematical model inherent in the dataset for the model to optimize and assuming that data adhere to Gaussian distribution models is inherently dangerous.

4.6.1 Gaussian Mixture Model

- A Gaussian mixture model is a probabilistic model that assumes all the data points are generated from a mixture of a finite number of Gaussian distributions with unknown parameters. Gaussian mixture model tends to group the data points belonging to a single distribution together.
- The advantage of mixture models is that they do not require which subpopulation a data point belongs to. It allows the model to learn the subpopulations automatically.
- Gaussian mixture models use the soft clustering technique for assigning data points to Gaussian distributions.
- Gaussian mixture model consists of two parts : Mean vectors and covariance matrices.
- A Gaussian distribution is defined as a continuous probability distribution that takes on a bell-shaped curve. Another name for Gaussian distribution is the normal distribution.
- In a one dimensional space, the probability density function of a Gaussian distribution is given by :

$$f(x|\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean and σ^2 is the variance

- Gaussian mixture models can be used for a variety of use cases, including identifying customer segments, detecting fraudulent activity and clustering images.

4.6.1.1 Expectation-Maximization Method for Parameter Estimation

- The EM algorithm is used for obtaining maximum likelihood estimates of parameters when some of the data is missing.

- The Expectation-Maximization (EM) algorithm is a broadly applicable approach to the iterative computation of Maximum Likelihood (ML) estimates. On each iteration of the EM algorithm, there are two steps, called the expectation step or the E-step and the maximization step or the M-step. Because of this, the algorithm is called the EM algorithm.
- The basic idea of the EM algorithm is to view the problem of computing the Maximum Likelihood Estimates (MLEs) from the data at hand as being computationally simpler if data had been observed also on some other random variables.
- Thus with the EM algorithm the observed data are viewed as being incomplete, and the problem is reformulated in a complete-data framework in which the ML solution may exist in closed form or may be amenable to ML computation with a standard computer package.
- Consider complete data-set consists of Z = (X;Y) but that only X is observed. The complete-data log likelihood is then denoted by $l(\theta;X;Y)$ where θ is the unknown parameter vector for which we wish to find the MLE.

  E-step : The E-step of the EM algorithm computes the excepted value of $l(\theta; X, Y)$ given observed data x and the current parameter estimate, θ_{old} say, In particular, we define.

$$Q(\theta;\theta_{old}) = E[l(\theta; X, Y | X, \theta_{old}]$$
$$= \int l(\theta; X, y)\, p(y | X, \theta_{old})\, dy$$

  Where $p(\cdot | x, \theta_{old})$ is the conditional density of y given the observed data, X and assuming $\theta = \theta_{old}$.

  M-step : The M-step consists of maximizing over θ the expectation computed in (1). That is, we set

$$\theta_{new} = \max_{\theta} Q(\theta;\theta_{old})$$

We then set $\theta_{old} = \theta_{new}$.

The two steps are repeated as necessary until the sequence of $\theta_{new's}$ converges.

- In summary, the expectation maximization algorithm alternates between the steps of guessing a probability distribution over completions of missing data given the current model (known as the E-step) and then re-estimating the model parameters using these completions (known as the M-step).

Review Question

1. *Discuss working of expectation maximization algorithm.*

4.7 Application of Clustering Technique

- Cluster analysis is a statistical technique used to group similar objects or data points into clusters or subgroups. It is an unsupervised learning method that aims to find inherent patterns or structures within the data without prior knowledge of the groupings.
- **Limitations :** Cluster analysis is a complex task and subject to certain limitations. It can be sensitive to the initial conditions, choice of distance measure and outliers in the data. Additionally, the results of cluster analysis should be interpreted carefully, as the effectiveness of the clustering depends on the quality of the data and the appropriateness of the chosen algorithm.

4.7.1 Market Segmentation

- Clustering techniques are used to group data/observations in a few segments so that data within any segment are similar while data across segments are different.
- Cluster analysis, in the context of market segmentation, is the application of a mathematical model to uncover groups of similar consumers based on the smallest variances among customers within each group.
- In market segmentation, the purpose of cluster analysis is to precisely categorize customers in order to create more successful customer marketing through personalization.

4.7.2 Social Network Analysis

- Clustering in social network analysis is not the same as traditional clustering. It necessitates classifying items based on their relationships as well as their properties.
- Traditional clustering algorithms group items only on their similarity and cannot be used for social network research.
- A social network clustering analysis technique, unlike typical clustering algorithms, can classify items in a social network based on their linkages and detect relationships between classes.

4.7.3 Image Segmentation

- Segmentation of images using clustering algorithms is a method for doing pixel-wise image segmentation.

- The clustering algorithm here aims to cluster the pixels that are close together in this form of segmentation. There are two ways to conduct segmentation via clustering : Merging clustering and divisive clustering

4.7.4 Anomaly Detection

- Clustering may be used to train the normalcy model by grouping comparable data points together into clusters using a distance function. Fig. 4.7.1 shows anomaly detection.

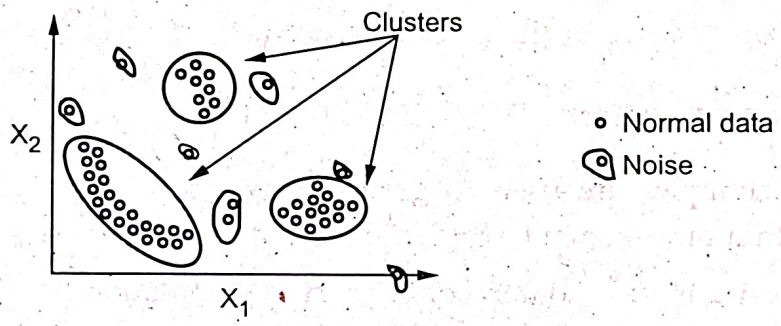


Fig. 4.7.1 Anomaly detection

- Clustering is appropriate for anomaly detection since no knowledge of the attack classes is required during training. Outliers in a dataset can be found using clustering and related approaches.
- Clustering can be used to find outliers or anomalies in the data that might point to mistakes or fraud. This may lessen the chance of suffering financial setbacks or other adverse effects.

❑❑❑

Unit V

5 Ensemble Learning

Syllabus

Ensemble Learning : *Introduction to Ensemble Learning, Need of Ensemble Learning, Homogeneous and Heterogeneous ensemble methods, Advantages and Limitations of Ensemble methods, Applications of Ensemble Learning.*

Basic Ensemble Learning Techniques : *Voting Ensemble, Types of Voting : Max Voting, Averaging, Weighted Average.*

Advanced Ensemble Learning Techniques :

Bagging : *Bootstrapping, Aggregation.*

Boosting : *Adaptive Boosting (AdaBoost), Gradient Boosting, XGBoost.*

Stacking : *Variance Reduction, Blending, Random Forest Ensemble, Advantages of Random Forest.*

Contents

5.1 Introduction to Ensemble Learning

5.2 Basic Ensemble Learning Techniques : Voting Ensemble

5.3 Advanced Ensemble Learning Techniques

5.4 Random Forest Ensemble

5.1 Introduction to Ensemble Learning

- Machine learning models suffer bias and/or variance. Bias is the difference between the predicted value and actual value by the model. Bias is introduced when the model doesn't consider the variation of data and creates a simple model. The simple model doesn't follow the patterns of data and hence the model gives errors in predicting training as well as testing data i.e. the model with high bias and high variance.
- Fig. 5.1.1 shows ensemble learning.

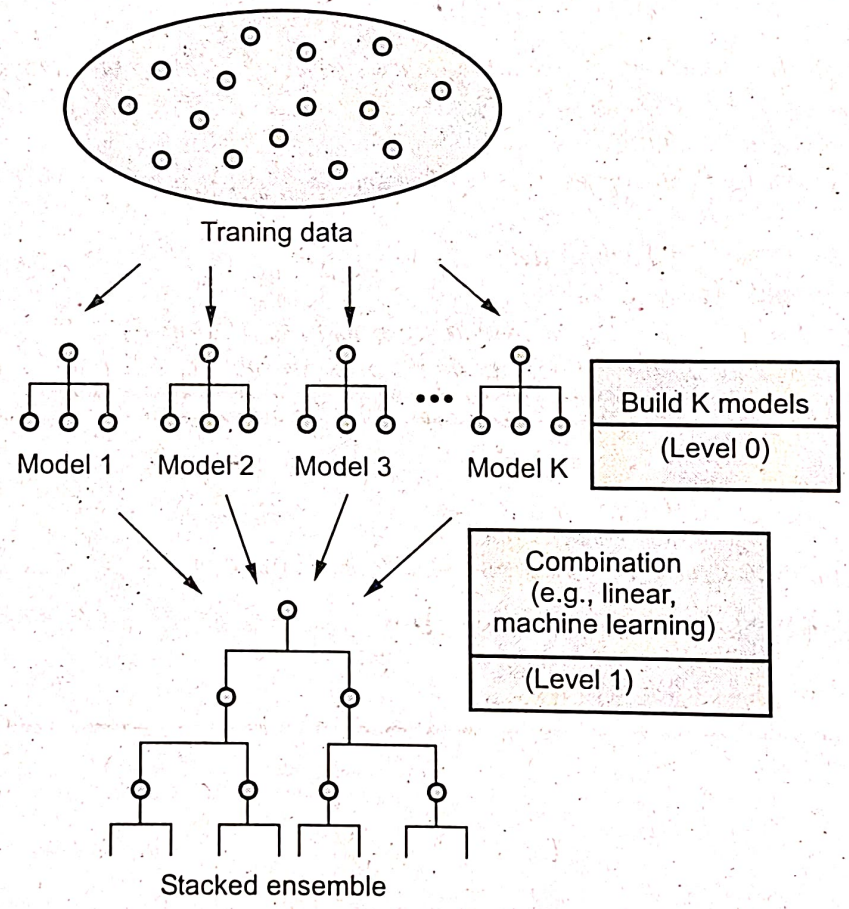


Fig. 5.1.1 Ensemble learning

- When the model follows even random quirks of data, as pattern of data, then the model might do very well on training dataset i.e. it gives low bias, but it fails on test data and gives high variance.
- Therefore, to improve the accuracy (estimate) of the model, ensemble learning methods are developed. Ensemble is a machine learning concept, in which several models are trained using machine learning algorithms. It combines low performing classifiers (also called as weak learners or base learner) and combine individual model prediction for the final prediction.

- The goal of ensemble methods is to combine the predictions of several base estimators built with a given learning algorithm in order to improve generalizability / robustness over a single estimator.
- **Why do ensemble methods work ?** Based on one of two basic observations :
  1. **Variance reduction :** If the training sets are completely independent, it will always help to average an ensemble because this will reduce variance without affecting bias, i.e. reduce sensitivity to individual data points.
  2. **Bias reduction :** For simple models, average of models has much greater capacity than single model. Averaging models can reduce bias substantially by increasing capacity and control variance by fitting one component at a time.

5.1.1 Need of Ensemble Learning

- When making decision trees, there are several factors we must take into consideration. But there is limit to add new questions in the decision tree. To solve this, ensemble learning method is used.
- Rather than just relying on one decision tree and hoping we made the right decision at each split, ensemble methods allow us to take a sample of decision trees into account, calculate which features to use or questions to ask at each split and make a final predictor based on the aggregated results of the sampled decision trees.
- Ensemble methods are techniques that create multiple models and then combine them to produce improved results. Ensemble methods in machine learning usually produce more accurate solutions than a single model would.
- Ensemble models use multiple machine learning algorithms to bring out better predictive results, as compared to using a single algorithm. There are different approaches in ensemble models to perform a particular task.
- There is another model called hybrid model that is flexible and helps to create a more innovative model than an ensemble model. While combining models we need to check how strong or weak a particular machine learning model is, to deal with a particular problem.

5.1.2 Homogeneous and Heterogeneous Ensemble Methods

- In machine learning, instead of building only a single model to predict target or future, how about considering multiple models to predict the target. This is the main idea behind ensemble learning.
- On the basis of type of base learners, ensemble methods can be categorized as homogeneous and heterogeneous ensemble methods. If base learners are same,

then it is a homogeneous ensemble method. If base learners are different then it is a heterogeneous ensemble method.

1. Homogeneous ensemble :

- Training only one learning algorithm on many different training sets obtained from the original training data to generate the base classifiers is known as **homogeneous ensemble.**
- Homogenous ensemble is a combination of the same type of classifiers. Even though the dataset consists of different classifiers, this ensemble technique makes a model that best suits a given problem. This type of technique is computationally expensive and is suitable for solving large datasets.
- "BAGGING" and "BOOSTING" are the popular methods that exhibit homogeneous ensemble.

2. Heterogeneous ensemble :

- In ensemble learning, training different learning algorithms on the original training set to generate the base classifiers is known as **heterogeneous ensemble.**
- Heterogeneous ensemble is a combination of different types of classifiers, in which each classifier is based on the same data. This method works on small datasets. "STACKING" comes in this category.

5.1.3 Advantages and Limitations of Ensemble Methods

1. Advantages :

- Improvement in predictive accuracy.
- This type of model is used to reduce bias from the individual base model having high bias.
- Better prediction than individual model.
- Ensembles can also be parallelized more easily, leading to more efficient performance.

2. Limitations :

- It is difficult to understand an ensemble of classifiers.
- High computation power is required as compared to individual models.
- Ensemble model is complex and can be more difficult to interpret.
- More time is consumed in training the ensemble models as compared to individual model.

5.2 Basic Ensemble Learning Techniques : Voting Ensemble

- The simplest way to combine multiple classifiers is by voting, which corresponds to taking a linear combination of the learners. Voting is an ensemble machine learning algorithm.
- For regression, a voting ensemble involves making a prediction that is the average of multiple other regression models.
- In classification, a hard voting ensemble involves summing the votes for crisp class labels from other models and predicting the class with the most votes. A soft voting ensemble involves summing the predicted probabilities for class labels and predicting the class label with the largest sum probability.
- Fig. 5.2.1 shows Base-learners with their outputs.
- In this methods, the first step is to create multiple classification/regression models using some training dataset. Each base model can be created using different splits of the same training dataset and same algorithm, or using the same dataset with different algorithms, or any other method.

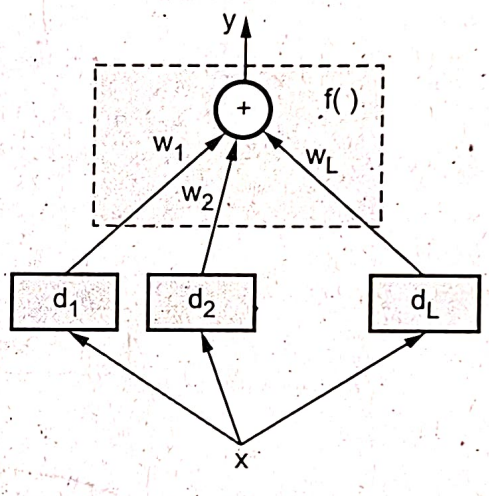


Fig. 5.2.1 Base-learners with their outputs

- Learn multiple alternative definitions of a concept using different training data or different learning algorithms. It combines decisions of multiple definitions, e.g. using weighted voting.
- Fig. 5.2.2 shows general idea of Base-learners with model combiner. (See Fig. 5.2.2 on next page)
- When combining multiple independent and diverse decisions each of which is at least more accurate than random guessing, random errors cancel each other out, and correct decisions are reinforced. Human ensembles are demonstrably better.
- Use a single, arbitrary learning algorithm but manipulate training data to make it learn multiple models.

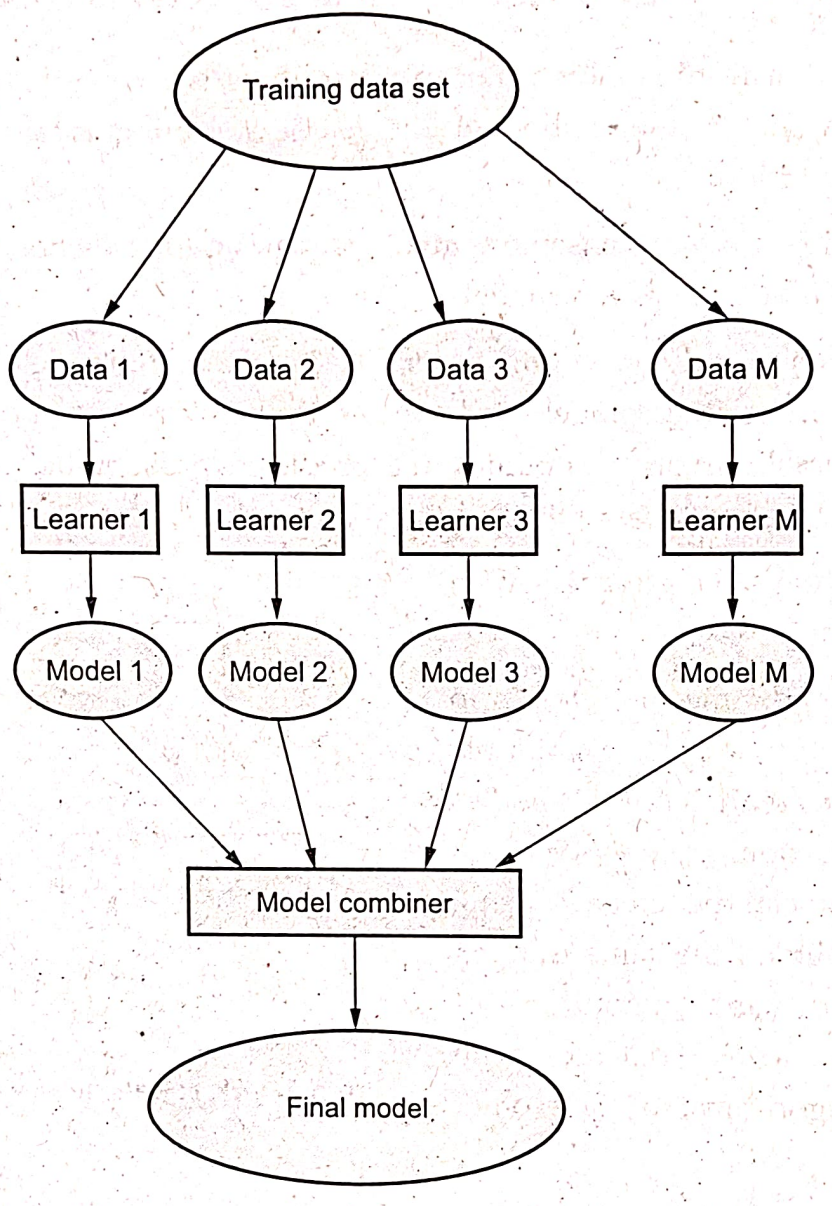


Fig. 5.2.2

5.2.1 Types of Voting

- Ensemble model types are as follows :
- Simple : Max Voting, Averaging, Weighted Averaging
- Advanced : Stacking, Blending, Bootstrap Sampling , Other are Bagging and Boosting

1. Max voting :

- The max voting method is generally used for classification problems. In this technique, multiple models are used to make predictions for each data point.
- The predictions by each model are considered as 'vote'. The predictions that we get from majority of the models are used as the final predictions.

- Considers the mode of all predictions. The result of max voting will be like :

Model-1	Model-2	Model-3	Model-4	Model-5	Final result
5	4	5	4	4	4

- Voting works because the opinion of the majority holds more weight than the vote on an individual. Max voting is used when we have discrete options on which the models can take a vote. The option that has the most number of votes is considered the chosen one.
- It is used for classification problems. Each machine learning model makes a vote, and the option with the maximum vote is the selected option.

2. Averaging voting :

- In simple averaging method, for every instance of test dataset, the average predictions are calculated.
- This method often reduces overfit and creates a smoother regression model. Thus the averaging method will take average of all values.

Model-1	Model-2	Model-3	Model-4	Model-5	Final result
5	4	5	4	4	4.4

3. Weighted average :

- Weighted averaging is a slightly modified version of simple averaging, where the prediction of each model is multiplied by the weight and then their average is calculated.
- All models are assigned different weights defining the importance of each model for prediction.
- For example : [(5*0.23) +(4*0.23) + (5*0.18) + (4*0.18) + (4*0.18)] = 4.41

	Model-1	Model-2	Model-3	Model-4	Model-5	Final result
Weights	0.23	0.23	0.18	0.18	0.18	
Predictions	5	4	5	4	4	4.41

5.3 Advanced Ensemble Learning Techniques

5.3.1 Bagging

- Bagging is also called bootstrap aggregating. Bagging and boosting are meta - algorithms that pool decisions from multiple classifiers. It creates ensembles by repeatedly randomly resampling the training data.

- Bagging was the first effective method of ensemble learning and is one of the simplest methods of arching. The meta - algorithm, which is a special case of the model averaging, was originally designed for classification and is usually applied to decision tree models, but it can be used with any type of model for classification or regression.
- Ensemble classifiers such as bagging, boosting and model averaging are known to have improved accuracy and robustness over a single model. Although unsupervised models, such as clustering, do not directly generate label prediction for each individual, they provide useful constraints for the joint prediction of a set of related objects.
- For given a training set of size n, create m samples of size n by drawing n examples from the original data, with replacement. Each *bootstrap sample* will on average contain 63.2 % of the unique training examples, the rest are replicates. It combines the m resulting models using simple majority vote.
- In particular, on each round, the base learner is trained on what is often called a "**bootstrap replicate**" of the original training set. Suppose the training set consists of n examples. Then a bootstrap replicate is a new training set that also consists of n examples, and which is formed by repeatedly selecting uniformly at random and with replacement n examples from the original training set. This means that the same example may appear multiple times in the bootstrap replicate, or it may appear not at all.
- It also decreases error by decreasing the variance in the results due to *unstable learners,* algorithms (like decision trees) whose output can change dramatically when the training data is slightly changed.
- **Pseudocode :**
  1. Given training data $(x_1, y_1), \ldots, (x_m, y_m)$
  2. For t = 1,..., T :
     a. Form bootstrap replicate dataset S_t by selecting m random examples from the training set with replacement.
     b. Let h_t be the result of training base learning algorithm on S_t.
  3. Output combined classifier :
     $H(x) = \text{majority}(h_1(x), \ldots, h_T(x))$.

Bagging steps :

1. Suppose there are N observations and M features in training data set. A sample from training data set is taken randomly with replacement.

2. A subset of M features is selected randomly and whichever feature gives the best split is used to split the node iteratively.
3. The tree is grown to the largest.
4. Above steps are repeated n times and prediction is given based on the aggregation of predictions from n number of trees.

Advantages of bagging :

1. Reduces over - fitting of the model.
2. Handles higher dimensionality data very well.
3. Maintains accuracy for missing data.

Disadvantages of bagging :

1. Since final prediction is based on the mean predictions from subset trees, it won't give precise values for the classification and regression model.

5.3.2 Boosting

- Boosting is a very different method to generate multiple predictions (function estimates) and combine them linearly. Boosting refers to a general and provably effective method of producing a very accurate classifier by combining rough and moderately inaccurate rules of thumb.
- Originally developed by computational learning theorists to guarantee performance improvements on fitting training data for a ***weak learner*** that only needs to generate a hypothesis with a training accuracy greater than 0.5. Final result is the weighted sum of the results of weak classifiers.
- A learner is weak if it produces a classifier that is only slightly better than random guessing, while a learner is said to be strong if it produces a classifier that achieves a low error with high confidence for a given concept.
- Revised to be a practical algorithm, AdaBoost, for building ensembles that empirically improves generalization performance. Examples are given weights. At each iteration, a new hypothesis is learned and the examples are reweighted to focus the system on examples that the most recently learned classifier got wrong.
- Boosting is a bias reduction technique. It typically improves the performance of a single tree model. A reason for this is that we often cannot construct trees which are sufficiently large due to thinning out of observations in the terminal nodes.
- Boosting is then a device to come up with a more complex solution by taking linear combination of trees. In presence of high - dimensional predictors, boosting

is also very useful as a regularization technique for additive or interaction modeling.

- To begin, we define an algorithm for finding the rules of thumb, which we call a weak learner. The boosting algorithm repeatedly calls this weak learner, each time feeding it a different distribution over the training data. Each call generates a weak classifier and we must combine all of these into a single classifier that, hopefully, is much more accurate than any one of the rules.
- Train a set of weak hypotheses : $h_1, ..., h_T$. The combined hypothesis H is a weighted majority vote of the T weak hypotheses. During the training, focus on the examples that are misclassified.

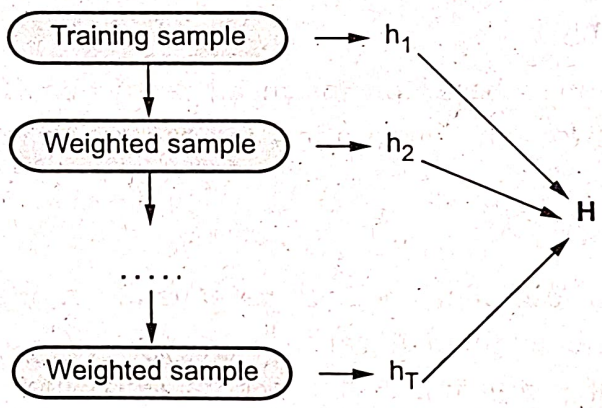


Fig. 5.3.1

AdaBoost :

- AdaBoost, short for "Adaptive Boosting", is a machine learning meta - algorithm formulated by Yoav Freund and Robert Schapire who won the prestigious "Gödel Prize" in 2003 for their work. It can be used in conjunction with many other types of learning algorithms to improve their performance.
- It can be used to learn weak classifiers and final classification based on weighted vote of weak classifiers.
- It is linear classifier with all its desirable properties. It has good generalization properties.
- To use the weak learner to form a highly accurate prediction rule by calling the weak learner repeatedly on different distributions over the training examples.
- Initially, all weights are set equally, but each round the weights of incorrectly classified examples are increased so that those observations that the previously classifier poorly predicts receive greater weight on the next iteration.
- **Advantages of AdaBoost :**
  1. Very simple to implement

2. Fairly good generalization
3. The prior error need not be known ahead of time.

Disadvantages of AdaBoost :

1. Suboptimal solution
2. Can over fit in presence of noise.

Boosting steps :

1. Draw a random subset of training samples d1 without replacement from the training set D to train a weak learner C1.
2. Draw second random training subset d2 without replacement from the training set and add 50 percent of the samples that were previously falsely classified/misclassified to train a weak learner C2.
3. Find the training samples d3 in the training set D on which C1 and C2 disagree to train a third weak learner C3.
4. Combine all the weak learners via majority voting.

Advantages of boosting :

1. Supports different loss function.
2. Works well with interactions.

Disadvantages of boosting :

1. Prone to over-fitting.
2. Requires careful tuning of different hyper - parameters.

5.3.3 Stacking

- Stacking, sometimes called stacked generalization, is an ensemble machine learning method that combines multiple heterogeneous base or component models via a meta-model.
- The base model is trained on the complete training data, and then the meta-model is trained on the predictions of the base models. The advantage of stacking is the ability to explore the solution space with different models in the same problem.
- The stacking based model can be visualized in levels and has at least two levels of the models. The first level typically trains the two or more base learners(can be heterogeneous) and the second level might be a single meta learner that utilizes the base models predictions as input and gives the final result as output. A

stacked model can have more than two such levels but increasing the levels doesn't always guarantee better performance.

- In the classification tasks, often logistic regression is used as a meta learner, while linear regression is more suitable as a meta learner for regression-based tasks.
- **Stacking** is concerned with combining multiple classifiers generated by different learning algorithms $L_1, \ldots, L_N$ on a single dataset S, which is composed by a feature vector $S_i = (x_i, t_i)$.
- The stacking process can be broken into two phases :
  1. Generate a set of base - level classifiers $C_1, \ldots, C_N$ where $C_i = L_i(S)$
  2. Train a meta - level classifier to combine the outputs of the base - level classifiers.
- Fig. 5.3.2 shows stacking frame.

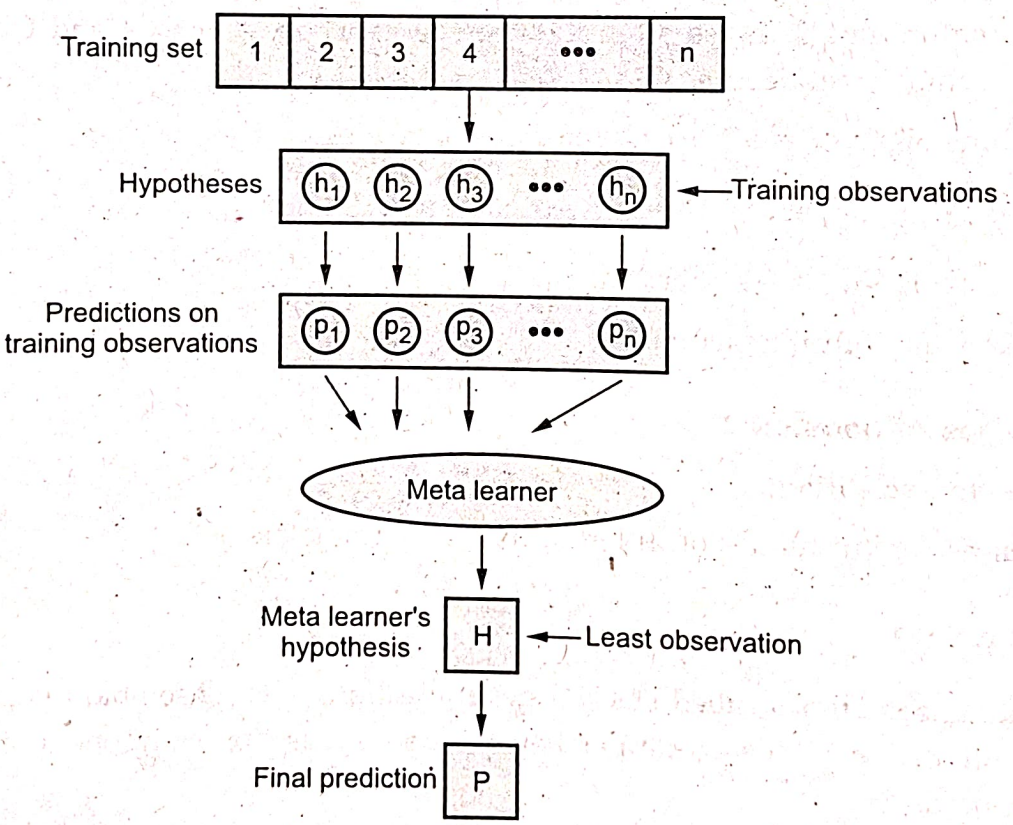


Fig. 5.3.2 Stacking frame

- The training set for the meta - level classifier is generated through a leave - one - out cross validation process.

$$\forall_i = 1, \ldots, n \text{ and } \forall_k = 1, \ldots, N : C_k^i$$

$$= L_k(S - s_i)$$

- The learned classifiers are then used to generate predictions for $s_i : \hat{y}_i^k = C_k^i(x_i)$

- The meta - level dataset consists of examples of the form $((\hat{y}_k^1, \ldots, \hat{y}_i^n), y_i)$, where the features are the predictions of the base - level classifiers and the class is the correct class of the example in hand.
- Why do ensemble methods work ?
- Based on one of two basic observations :
  1. **Variance reduction :** If the training sets are completely independent, it will always helps to average an ensemble because this will reduce variance without affecting bias (e.g. - bagging) and reduce sensitivity to individual data points.
  2. **Bias reduction :** For simple models, average of models has much greater capacity than single model Averaging models can reduce bias substantially by increasing capacity and control variance by Citting one component at a time.

5.3.4 AdaBoost

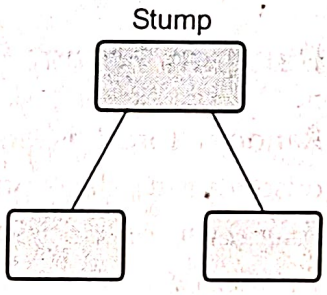


Fig. 5.3.3 AdaBoost

- AdaBoost also referred to as adaptive boosting is a method in Machine Learning used as an ensemble method. The maximum not unusual algorithm used with AdaBoost is selection trees with one stage meaning with decision trees with most effective 1 split. These trees also are referred to as decision stumps.
- The working of the AdaBoost version follows the beneath-referred to path :
  - Creation of the base learner.
  - Calculation of the total error via the beneath formulation.
  - Calculation of performance of the decision stumps.
  - Updating the weights in line with the misclassified factors.

Creation of a new database :

AdaBoost ensemble :

- In the ensemble approach, we upload the susceptible fashions sequentially and then teach them the use of weighted schooling records.
- We hold to iterate the process till we gain the advent of a pre-set range of vulnerable learners or we can not look at further improvement at the dataset. At the end of the algorithm, we are left with some vulnerable learners with a stage fee.

5.3.5 Difference between Bagging and Boosting

Sr. No.	Bagging	Boosting
1.	Bagging is a technique that builds multiple homogeneous models from different subsamples of the same training dataset to obtain more accurate predictions than its individual models	Boosting refers to a group of algorithms that utilize weighted averages to make weak learning algorithms stronger learning algorithms.
2.	Learns them independently from each other in parallel	Learns them sequentially in a very adaptative way
3.	It helps in reducing variance.	It helps in reducing bias and variance.
4.	Every model receives an equal weight.	Models are weighted by their performance.

5.4 Random Forest Ensemble

- Random forest is an ensemble machine learning algorithm. Random forest is an ensemble of decision tree algorithms. It is an extension of bootstrap aggregation (bagging) of decision trees and can be used for classification and regression problems.
- Random forest algorithm is a supervised machine learning algorithm.

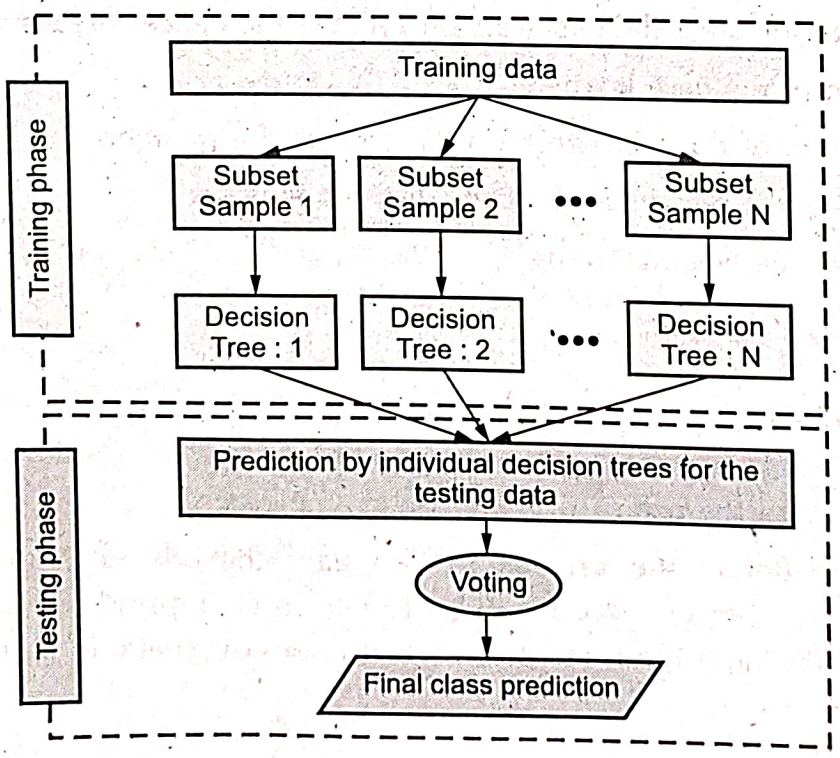


Fig. 5.4.1 Working principle of the random forest classifier

- One big advantage of random forest is that it can be used for both classification and regression problems, which form the majority of current machine learning systems.
- Fig. 5.4.1 (Fig. 5.4.1 see on previous page) shows the working principle of the RF classifier.
- The training algorithm applies the general technique of bootstrap aggregating or bagging. Given a training set $X = \{x_1, \ldots, x_n\}$ with responses $Y = \{y_1, \ldots, y_n\}$, bagging selects a random sample with replacement of the training set repeatedly for B times and fits decision trees to these samples.
- After training, the class prediction for an unseen sample (x'), is determined by taking a majority vote as follows :

$$C(x') = \arg\max \sum_{j=1}^{B} = W_j I(h_j(x') = i)$$

where $W_1, \ldots, W_B$ are weights that sum to 1, which are usually set to 1/B; $I(\cdot)$ is an indicator function; and $h_j(\cdot)$ is the prediction function for the j^{th} decision tree.

- The following steps explain the working random forest algorithm:

Step 1 : Select random samples from a given data or training set.

Step 2 : This algorithm will construct a decision tree for every training data.

Step 3 : Voting will take place by averaging the decision tree.

Step 4 : Finally, select the most voted prediction result as the final prediction result.

- **Features of Random Forest**

1. Each tree has a unique attribute, variety and features concerning other trees. Not all trees are the same.
2. **Immune to the curse of dimensionality :** Since a tree is a conceptual idea, it requires no features to be considered. Hence, the feature space is reduced.
3. **Parallelization :** We can fully use the CPU to build random forests since each tree is created autonomously from different data and features.
4. **Train-Test split :** In a random forest, we don't have to differentiate the data for train and test because the decision tree never sees 30 % of the data.
5. **Stability :** The final result is based on bagging, meaning the result is based on majority voting or average.

- **When we use Random Forest ?**

1. Decision tree algorithms easily lead to overfitting problems. Random forest algorithm can overcome this.

2. Capable of both regression and classification problems.
3. Handle a large number of features.
4. Estimating which features are important in the underlying data being modelled.
5. Random forest is capable of learning without carefully crafted data transformations.
6. Output probabilities for classification problems.

How random forest algorithm can be build using decision tree ?

- A random forest is simply a collection of decision trees whose results are aggregated into one final result. Their ability to limit overfitting without substantially increasing error due to bias is why they are such powerful models.
- One way random forests reduce variance is by training on different samples of the data. A second way is by using a random subset of features. A random forest is a supervised machine learning algorithm that is constructed from decision tree algorithms.
- A random forest algorithm consists of many decision trees. The 'forest' generated by the random forest algorithm is trained through bagging or bootstrap aggregating. Bagging is an ensemble meta-algorithm that improves the accuracy of machine learning algorithms.
- Decision trees are the building blocks of a random forest algorithm. A decision tree is a decision support technique that forms a tree-like structure.
- A decision tree consists of three components : decision nodes, leaf nodes and a root node. A decision tree algorithm divides a training dataset into branches, which further segregate into other branches. This sequence continues until a leaf node is attained. The leaf node cannot be segregated further.
- Entropy and information gain are the building blocks of decision trees. The information gain concept involves using independent variables (features) to gain information about a target variable (class). The entropy of the target variable (Y) and the conditional entropy of Y (given X) are used to estimate the information gain.
- Information gain is used in the training of decision trees. The random forest employs the bagging method to generate the required prediction.
- Bagging involves using different samples of data (training data) rather than just one sample. A training dataset comprises observations and features that are used for making predictions. The decision trees produce different outputs, depending on the training data fed to the random forest algorithm. These outputs will be ranked and the highest will be selected as the final output.

- Classification in random forests employs an ensemble methodology to attain the outcome. The training data is fed to train various decision trees. This dataset consists of observations and features that will be selected randomly during the splitting of nodes.
- Fig. 5.4.2 shows random forest classifier using decision tree.

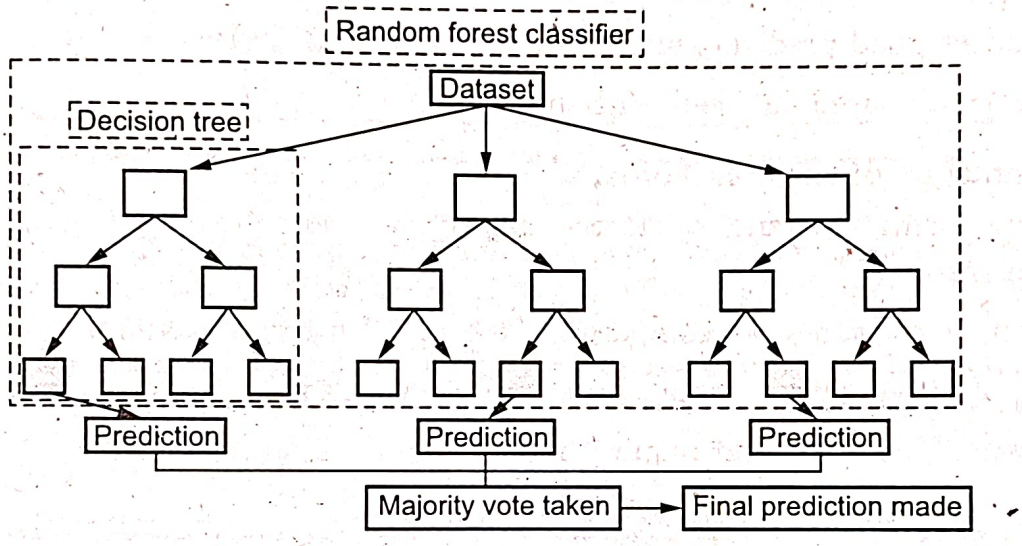


Fig. 5.4.2 Random forest classifier using decision tree

- Every decision tree consists of decision nodes, leaf nodes and a root node. The leaf node of each tree is the final output produced by that specific decision tree. The selection of the final output follows the majority-voting system.
- In this case, the output chosen by the majority of the decision trees becomes the final output of the rain forest system.

5.4.1 Applications of Random Forest

- **Banking :** It predicts a loan applicant's solvency. This helps lending institutions make a good decision on whether to give the customer loan or not. They are also being used to detect fraudsters.
- **Health care :** Health professionals use random forest systems to diagnose patients. Patients are diagnosed by assessing their previous medical history. Past medical records are reviewed to establish the proper dosage for the patients.
- **Stock market :** Financial analysts use it to identify potential markets for stocks. It also enables them to remember the behaviour of stocks.
- **E-commerce :** Through this system, e-commerce vendors can predict the preference of customers based on past consumption behaviour.

5.4.2 Advantages and Disadvantages of Random Forest

1. Advantages of Random Forest

- It reduces the risk of overfitting and the required training time.
- It offers a high level of accuracy.
- It can perform both regression and classification tasks.
- Produces good predictions that can be understood easily.
- Can handle large data sets efficiently.

2. Disadvantages of Random Forest

- While using a random forest algorithm, more resources are required for computation.
- It consumes more time compared to the decision tree algorithm.
- Less intuitive when we have an extensive collection of decision trees.
- Extremely complex and requires more computational resources.

❑❑❑

Unit VI

6 Reinforcement Learning

Syllabus

Reinforcement learning : *What is Reinforcement Learning ? Need for Reinforcement Learning, Supervised vs Unsupervised vs Reinforcement Learning, Types of Reinforcement, Elements of Reinforcement Learning, Real time applications of Reinforcement learning.*

Markov's Decision Process : *Markov property, Markov chain/process, Markov reward process (MRP), Markov decision process (MDP), Return, Policy, Value functions, Bellman equation.*

Q Learning : *Introduction of Q-Learning, Important terms in Q learning, Q table, Q functions, Q learning algorithm.*

Contents

6.1 What is Reinforcement Learning ?

6.2 Types of Reinforcement

6.3 Positive Vs Negative Reinforced Learning

6.4 Markov Decision Process

6.5 Q Learning

6.1 What is Reinforcement Learning ?

- Reinforcement Learning (RL) is a type of machine learning technique that enables an agent to learn in an interactive environment by trial and error using feedback from its own actions and experiences.
- Reinforcement learning is the science of decision making. It is about learning the optimal behavior in an environment to obtain maximum reward. This optimal behaviour is learned through interactions with the environment and observations of how it responds.
- Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions and rewards.
- Reinforcement learning is an approach to machine learning in which the agents are trained to make a sequence of decisions. The agent, also called as an AI agent gets trained in the following manner :
  a) The agent interacts with the environment and make decisions or choices. For training purpose, the agent is provided with the contextual information about the environment and choices.
  b) The agent is provided with the feedback or rewards based on how well the action taken by the agent or the decision made by the agent resulted in achieving the desired goal.
- User will get immediate feedback in supervised learning and no feedback from unsupervised learning. But in the reinforced learning, you will get delayed scalar feedback.
- Reinforcement learning is learning what to do and how to map situations to actions. The learner is not told which actions to take. Fig. 6.1.1 shows concept of reinforced learning.

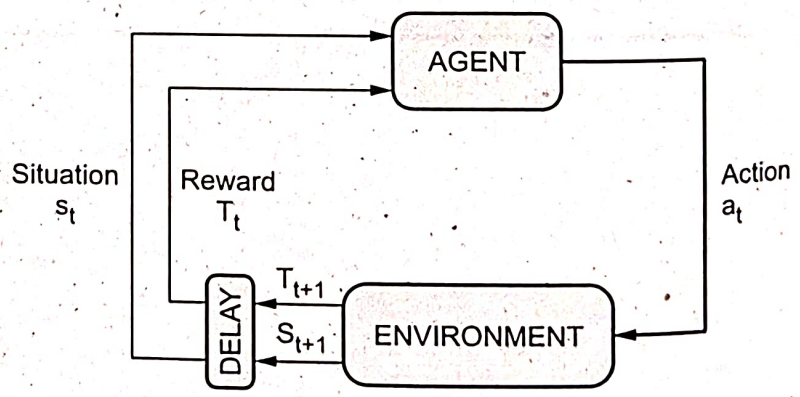


Fig. 6.1.1 Reinforced learning

- Reinforced learning is deals with agents that must sense and act upon their environment. It combines classical Artificial Intelligence and machine learning techniques.
- It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance. Simple reward feedback is required for the agent to learn its behavior; this is known as the reinforcement signal.

- Two most important distinguishing features of reinforcement learning is trial-and-error and delayed reward.
- With reinforcement learning algorithms an agent can improve its performance by using the feedback it gets from the environment. This environmental feedback is called the reward signal.
- Based on accumulated experience, the agent needs to learn which action to take in a given situation in order to obtain a desired long term goal. Essentially actions that lead to long term rewards need to reinforced. Reinforcement learning has connections with control theory, Markov decision processes and game theory.
- **Example of reinforcement learning :** A mobile robot decides whether it should enter a new room in search of more trash to collect or start trying to find its way back to its battery recharging station. It makes its decision based on how quickly and easily it has been able to find the recharger in the past.

6.1.1 Elements of Reinforcement Learning

- Reinforcement learning elements are as follows :
  1. Policy
  2. Reward function
  3. Value function
  4. Model of the environment
- Fig. 6.1.2 shows elements of RL.
- **Policy :** Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.
- **Reward function :** Reward function is used to define a goal in a reinforcement learning problem. It also maps each perceived state of the environment to a single number.

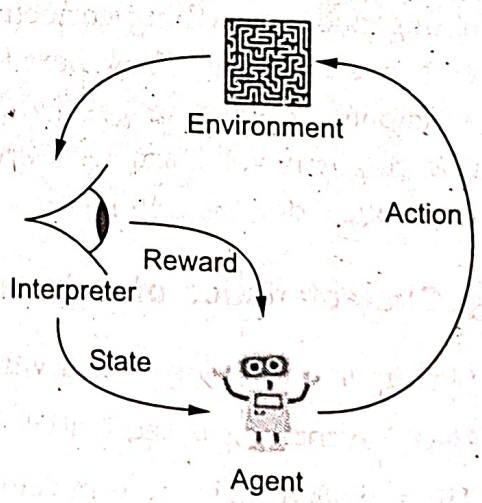


Fig. 6.1.2 Elements of reinforcement learning

- **Value function :** Value functions specify what is good in the long run. The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.
- **Model of the environment :** Models are used for planning.
- **Credit assignment problem :** Reinforcement learning algorithms learn to generate an internal value for the intermediate states as to how good they are in leading to the goal.

- The learning decision maker is called the agent. The agent interacts with the environment that includes everything outside the agent.
- The agent has sensors to decide on its state in the environment and takes an action that modifies its state.
- The reinforcement learning problem model is an agent continuously interacting with an environment. The agent and the environment interact in a sequence of time steps. At each time step t, the agent receives the state of the environment and a scalar numerical reward for the previous action, and then the agent then selects an action.
- Reinforcement learning is a technique for solving Markov decision problems.
- Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards. This framework is intended to be a simple way of representing essential features of the artificial intelligence problem.
- Example of the general Reinforcement Learning concept : Imagine teaching a dog some new tricks. Unfortunately, dogs do not understand human speech, so we cannot directly tell pet what to do. Therefore, we will imitate the situation and dog will try to act one way or another.
- If dog does everything correctly, for example, sit down after a "sit" command, we will give it a treat. Thus, next time in a similar situation a dog will again act as an anticipated hoping to get a treat again. However, if we do not use a treat at all, dog might never learn new tricks at all as it will not receive any positive stimuli from our side.

6.1.2 Characteristics of Reinforced Learning

- No supervision, only a real value or reward signal
- Decision making is sequential
- Time plays a major role in reinforcement problems
- Feedback isn't prompt but delayed

6.1.3 Exploration Vs Exploitation

- Exploitation : make the best decision given current information
- Exploration : gather more information

1. E3 Algorithm (Explicit Exploration)

- The E3 algorithm's approach to solving the exploration versus exploitation dilemmas is to use an explicit exploration model that uses direct and planned exploration.
- An agent will directly explore, by performing the action it has tried in that state the fewest amount of times in that particular state, until a state is known well enough where the greatest rewards for that state can be approximated causing planned exploration.
- When E3 encounters a known state it first attempts to find a near optimal action to exploit based on values returned to it by the value function and if an optimal action is not found the most well-known state action pairs are set to return a reward of zero and the remaining actions are set to return an optimal value temporarily in the reward function, to occur.

2. R-Max Algorithm (Implicit Exploration)

- The R-Max algorithm approach uses an implicit exploration model which utilizes the idea of optimistic initial values. That is, as an initial condition the agent will assume that every possible action in every state provides the optimal reward.
- This implicit approach requires the agent begin with some preconceived model of the environment that is updated as it encounters various state changes by performing actions.
- It is not required that this preconceived model need accurately represent the environment; it only need be some approximation that the agent can adapt.
- The environment model is updated in very much the same manner as E3 by recording all state action pairs, K(s, a).
- **Example :**
  1. Restaurant Selection
     a) Exploitation : Go to favorite restaurant
     b) Exploration : Try a new restaurant
  2. Online Banner Advertisements
     a) Exploitation : Show the most successful advert
     b) Exploration : Show a different advert
  3. Oil Drilling
     a) Exploitation : Drill at the best known location
     b) Exploration : Drill at a new location

4. Game Playing
   a) Exploitation : Play the move you believe is best
   b) Exploration : Play an experimental move
5. Clinical Trial
   a) Exploitation : Choose the best treatment so far
   b) Exploration : Try a new treatment

6.1.4 Rewards and Penalties

- Reinforcement learning is training by rewards and punishments. Reward-related (positivity) and punishment-related (negativity) systems have been viewed as separate, in biological terms, implemented by appetitive and aversive components of the nervous system but computational modeling of the relationship between such reward and punishment systems is relatively lacking.
- Where such modeling exists, typically, it concerns the additive combining of external reward and punishment signals conflated into a single measure of value: Punishment is viewed as a type of negative reward added to the positive reward value.
- If the rewards and the result of actions are not deterministic, then we have a probability distribution for the reward from which rewards are sampled, and there is a probability distribution for the next state.
- An agent's reward function determines the perception of a received reward. It determines whether a reward was 'good' or 'bad' by comparing it other rewards received from the same state and also to rewards received from other states.
- In the simple coin flip game, heads would be perceived by the reward function as the return of a 'good' reward and that tails would be perceived as a 'bad' reward. However, the reward function might weight the return of tails depending on the state it appeared.
- A tails returned at step one might return a 'worse' reward than a tails received at step two after receiving a heads at step one because the probability of receiving an optimal reward is greatly diminished.

6.1.5 Application

1. A master chess player makes a move. The choice is informed both by planning--anticipating possible replies and counter replies and by immediate, intuitive judgments of the desirability of particular positions and moves.
2. An adaptive controller adjusts parameters of a petroleum refinery's operation in real time. The controller optimizes the yield/cost/quality trade-off on the basis of

specified marginal costs without sticking strictly to the set points originally suggested by engineers.

3. A mobile robot decides whether it should enter a new room in search of more trash to collect or start trying to find its way back to its battery recharging station. It makes its decision based on how quickly and easily it has been able to find the recharger in the past.

6.1.6 Difference between Supervised, Unsupervised and Reinforced Learning

Supervised learning	Unsupervised learning	Reinforcement learning
Supervised learning requires that the target variable is well defined and that a sufficient number of its values are given.	For unsupervised learning typically either the target variable is unknown or has only been recorded for too small a number of cases.	Reinforcement learning is learning what to do and how to map situations to actions. The learner is not told which actions to take.
Supervised learning deals with two main tasks regression and classification.	Unsupervised learning deals with clustering and associative rule mining problems.	Reinforcement Learning deals with exploitation or exploration, Markov's decision processes, Policy Learning, Deep Learning and value learning.
The input data in supervised learning in labelled data.	Unsupervised learning uses unlabelled data.	The data is not predefined in reinforcement learning.
Learns by using labelled data.	Trained using unlabelled data without any guidance.	Works on interacting with the environment.
Maps the labeled inputs to the known outputs.	Understands patterns and discovers the output.	Follows the trial and error method.

6.2 Types of Reinforcement

- Fig. 6.2.1 shows types of Reinforced learning algorithm.
(See Fig. 6.2.1 on next page)
- In a model - based RL environment, the policy is based on the use of a machine learning model. When an RL environment or system utilizes the use of machine learning models like random forest, gradient boost, neural networks and others, such an RL system is model - based.

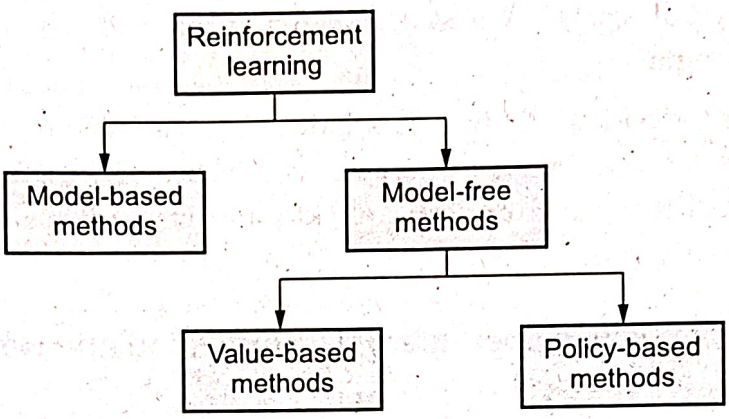


Fig. 6.2.1 Types of Reinforced learning algorithm

- With a model - free RL, such a system has no policy based on the use of machine learning models; its policy is guided by the use of non - ML algorithms.

1. Value - based :

- The main goal of this method is to maximize a value function. Here, an agent through a policy expects a long - term return of the current states.
- Value functions are functions of states or of state - action pairs, that estimate how good it is for an agent to be in a given state or how good it is for the agent to perform a given action in a given state.

2. Policy - based :

- In policy - based, user enable to come up with a strategy that helps to gain maximum rewards in the future through possible actions performed in each state. Two types of policy - based methods are deterministic and stochastic.
- Deterministic : In this sub-division, the same action is produced by the policy at any state.
- Stochastic : Here, probability determines the action produced.

3. Model - based :

- In this method, we need to create a virtual model for the agent to help in learning to perform in each specific environment. Model - based reinforcement learning methods rely on the use of an explicit parameterization of the transition model.

6.3 Positive Vs Negative Reinforced Learning

- There are two types of reinforcement learning methods : Positive reinforcement and Negative reinforcement.

1. Positive reinforcement

- Positive reinforcement learning is the process of encouraging or adding something when an expected behavior pattern is exhibited to increase the likelihood of the same behavior being repeated.

- For example, if a child passes a test with impressive grades, they can be positively reinforced with an ice cream cone.

2. Negative reinforcement

- Negative reinforcement involves increasing the chances of specific behavior to occur again by removing the negative condition.
- For example, if a child fails a test, they can be negatively reinforced by taking away their video games. This is not precisely punishing the child for failing, but removing a negative condition (in this case, video games) that might have caused the kid to fail the test.
- Basically, when a person's behaviour leads to the removal of something that was unpleasant to that person then negative reinforcement is occuring.
- **Example :** *An HR who wishes to increase productivity among employees.*
- Now applying the two REINFORCEMENTs here to increase the likelihood of the desired behavior that is - Increased productivity.

a) Positive reinforcement : Rewarding the employees who reach the given target with extra paid leaves.

- Here : The employee will work towards the target as he is motivated to get more paid leaves.

b) Negative reinforcement : Reducing office noise and interference in privacy by giving more space to employees i.e., reducing crowding and offering more open space, if the targets are met and productivity is increased.

- Here : To avoid the crowd, noises and privacy invasion, the employees will work towards achieving the targets. Hence, productivity is increased.
- Thus, both reinforcements are increasing the chances of the desired behavior (Increased productivity).

6.4 Markov Decision Process

- A reinforcement learning task that satisfies the Markov property is called a Markov Decision Process (MDP). If the state and action spaces are finite, then it is called a finite Markov Decision Process (finite MDP). Finite MDPs are particularly important to the theory of reinforcement learning.
- In a Markov Decision Process (MDP) the agent can perceive a set S of distinct states of its environment and has a set A of actions that it can perform.
- A Markov decision process is a tuple $(S, A, \{P_{sa}\}, \gamma, R)$.

Where :

- S is a set of actions. (For example, in autonomous helicopter flight. S might be the set of all possible positions and orientation of the helicopter).

- A is set of actions, (For example, the set of all possible directions in which you can push the helicopter's control sticks).
- P_{sa} are the state transition probabilities. For each state $s \in S$ and action $a \in A$, P_{sa} is a distribution over the state space. We'll say more about this layer, but briefly, P_{sa} gives the distribution over what states we will transition to if we take action a in state s.
- $\gamma \in (0,1)$ is called the **discount factor.**
- $R : S \times A \rightarrow \Re$ is the **reward function.** (Rewards are sometimes also written as a function of a state S only in which case we would have $R : S \rightarrow \Re$.

6.4.1 Markov Property

- If our state representation is as effective as having a full history, then we say that our model fulfills the requirements of the Markov property.
- To illustrate this with an example, think of playing Tic-Tac-Toe. When we are able to take a decision based on the current state, rather than needing to know the whole history, then we say that we satisfy the conditions of the Markov property.
- A state S_t is Markov if and only if

  $P[S_{t+1}|S_t] = P[S_{t+1}|S_1, \ldots, S_t]$
- The state captures all relevant information from the history. Once the state is known, the history may be thrown away, i.e. the state is a sufficient statistic of the future.
- For a Markov state (s) and successor state (s'), the state transition probability is defined by

  $$p_{ss'} = P[S_{t+1} = s' | S_t = s]$$
- State transition matrix p defines transition probabilities from all states (s) to all successor states (s'),

$$P = \text{from} \begin{bmatrix} & \text{to} & \\ p_{11} & \cdots & p_{1n} \\ \vdots & & \\ p_{n1} & \cdots & p_{nn} \end{bmatrix}$$

where each row of the matrix sums to 1.

- **Example :** Given that a student last drink purchase was tea, there is a 90 % chance that his next drink purchase will also be tea. If a student last drink purchase was milk, there is an 80 % chance that his next drink purchase will also be milk. What is the probability that he will purchase tea two purchases from now ?

$$P = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$$

State diagram :

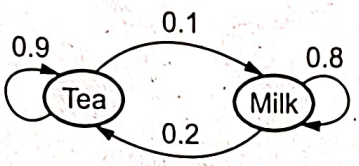


Fig. 6.4.1

$$P^2 = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix} \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$$

$$= \begin{bmatrix} 0.9\times0.9+0.1\times0.2 & 0.9\times0.1+0.1\times0.8 \\ 0.2\times0.9+0.8\times0.2 & 0.2\times0.1+0.8\times0.8 \end{bmatrix}$$

$$P^2 = \begin{bmatrix} 0.83 & 0.17 \\ 0.34 & 0.66 \end{bmatrix}$$

Probability that student will purchase tea two purchase from now

$$= 0.2\times0.9+0.8\times0.2$$

$$= 0.34$$

6.4.2 Markov Chain / Process

- A Markov process is a memoryless random process, i.e. a sequence of random states S_1; S_2; ... with the Markov property.
- Markov process or Markov chain is a tuple <S; P>
  where S is a finite set of states

  P is a state transition probability matrix
- Markov chain is characterized by three components :
  1. State space
  2. Initial distribution
  3. Markov kernel (transition)
- State space is the range of all random variables. The initial distribution quantifies the starting configuration for the chain.
- Kernel quantifies the probability of transitioning from state x_i to x_j so it establishes how the chain evolves.

Classification of states of a Markov chain :

1. State i leads to state j ($i \rightarrow j$) :

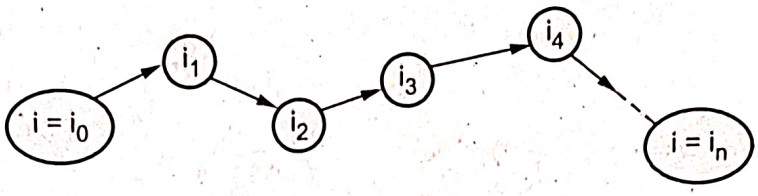


Fig. 6.4.2

2. State i and state j communicate ($i \leftrightarrow j$) :

   if $i \rightarrow j$ and $j \rightarrow i$

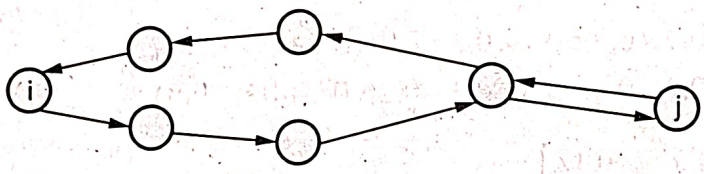


Fig. 6.4.3

3. A state i of a Markov chain is called a return chain if $p_{i,j}^{(n)} > 0$ for some $n \geq 1$.
4. A state i is said to be an absorbing state if and only if $p_{ij} = 1$.
5. The Markov chain is said to be 'irreducible' if there is only one class, i.e. all state communicate with each other.
6. A state with period 1 is said to be aperiodic.
7. The Markov chain does not drift away to infinity.
8. A Markov chain absorbing if it has at least one absorbing state and if from every state it is possible to go to an absorbing state.
9. A Markov chain is called a regular chain if some power of the transition matrix has only positive elements.
10. If $i \leftrightarrow j$ and $j \leftrightarrow k$ then $i \leftrightarrow k$.

- Irreducible means that every state is accessible from every other state. Aperiodic means that there exists at least one state to itself is possible.
- Positive recurrent means that for every state, the expected return time is finite. If the Markov chain is positive recurrent, there exist a stationary distribution.
- Communication is an equivalence relation : The states can be grouped into equivalent classes so that within each class all the states communicate with each other and two states from two different classes never communicate with each. The

equivalence classes defined by the relation $\leftrightarrow$ are called the irreducible classes of states.

- A Markov chain with a state space which is an irreducible class is called irreducible.

6.4.3 Markov Reward Process (MRP)

- The Markov Reward Process is an extension on the original Markov Process, but with adding rewards to it.
- A Markov Reward Process is a tuple $< S; P; R; \gamma >$

  Where S is a finite set of states

  P is a state transition probability matrix

  R is a reward function

  γ is a discount factor

6.4.4 Value Function

- A value function is the long-term value of a state or an action. In other words, it's the expected return over a state or an action.
- Because we want to find a policy which maximizes the expected return, it is a good idea to estimate the expected return. Then we can search through the space of policies for a good policy.
- Value functions represent the expected return, for every state, given a certain policy. Computing value functions is an intermediate step towards computing good policies.

State value function :

- The state value function of a policy Π is a function $V^{\Pi} : S \rightarrow R$
- The value of state under policy is the expected return if the agent starts from state and picks actions according to policy :

  $V^{\pi}(s) = E_{\pi}[R_t | S_t = s]$

- For a finite state space, we can represent this as an array, with one entry for every state.

6.4.5 Bellman Equation

- The Bellman equation is central to Markov decision processes. It outlines a framework for determining the optimal expected reward at a state by answering the question : "what is the maximum reward an agent can receive if they make the optimal action now and for all future decisions" ?

- The Bellman equation gives a standard representation for value functions. It decomposes the value function into two components :
  1. The immediate reward
  2. Discounted value of the future state
- By putting all the previous pieces together, we get :

$$V^{\pi}(s) = \sum_{a \in A} \pi(s,a) \left(r_a(s) + \gamma \sum_{s' \in S} T_a(s,s') V^{\pi}(s') \right)$$

- This is a system of linear equations (one for every state) whose unique solution is V^{π}.
- The uniqueness is ensured under mild technical conditions on the transitions p.

6.5 Q Learning

- Q-learning is a reinforcement learning techniques used in machine learning. The goal of Q-learning is to learn a policy, which tells an agent which action to take under which circumstances.
- In Q-learning, an agent tries to learn the optimal policy from its history of interaction with the environment.
- It has the ability to compute the utility of the actions without a model for the environment. It takes the help of action-value pair and the expected reward from the current action.
- Q-learning is a model-free, value-based, off-policy algorithm that will find the best series of actions based on the agent's current state. The "Q" stands for quality. Quality represents how valuable the action is in maximizing future rewards.
- Model-free algorithms learn the consequences of their actions through the experience without transition and reward function.
- The value-based method trains the value function to learn which state is more valuable and take action. On the other hand, policy-based methods train the policy directly to learn which action to take in a given state. In the off-policy, the algorithm evaluates and updates a policy that differs from the policy used to take an action.
- An idea behind Q-learning is that it is possible to collect data samples (x_t, u_t, r_t, x_{t+1}) and over time learn what the optimal Q-function is $Q^*(x, u)$.
- For simplicity an infinite-horizon ($T = \infty$) problem will be considered, such that the optimal value and Q-functions will be stationary and in particular the optimal Q-function will satisfy the equation.

- Even if the model and reward function are unknown, for a given sample (x_t, u_t, r_t, x_{t+1}) that is collected by interacting with the environment, the expectation :

  $$E[(r_t + \gamma \max_{u' \in u} Q^*(x_{t+1}, u')) - Q^*(x_t, u_t)] = 0,$$

  will hold for the optimal Q-function. Therefore, the idea of Q-learning is that an approximation of the optimal Q-function can be improved over time by collecting data and trying to ensure that the above conditions holds.
- Q-learning is referred to as a model-free method because it forgoes explicitly estimating the true system dynamics, and directly estimates the Q-function. It is also called a value-based model-free method since it does not directly build the policy, but rather estimates the optimal Q-function to implicitly define the policy.
- Consider a computational agent moving around some discrete, finite world, choosing one from a finite collection of actions at every time step. The world constitutes a controlled Markov process with the agent as a controller.
- At step n, the agent is equipped to register the state $x_n (\in X)$ of the world, an can choose its action an $(\in 2)$ accordingly.
- The agent receives a probabilistic reward r_n, whose mean value depends only on the state and action and the state of the world changes probabilistically to y_n according to the law :

  $\text{Prob}[y_n = y | x_n, a_n] = P_{x_n y}[a_n]$
- The task facing the agent is that of determining an optimal policy, one that maximizes total discounted expected reward.

6.5.1 Important Terms in Q Learning

- Terminology used are as follows :
  1. **States(s) :** The current position of the agent in the environment.
  2. **Action(a) :** A step taken by the agent in a particular state.
  3. **Rewards :** For every action, the agent receives a reward and penalty.
  4. **Episodes :** The end of the stage, where agents can't take new action. It happens when the agent has achieved the goal or failed.
  5. **$Q(S_{t+1}, a)$:** Expected optimal Q-value of doing the action in a particular state.
  6. **$Q(S_t, A_t)$:** It is the current estimation of $Q(S_{t+1}, a)$.
  7. **Q-Table :** The agent maintains the Q-table of sets of states and actions.
  8. **Temporal Differences (TD) :** Used to estimate the expected value of $Q(S_{t+1}, a)$ by using the current state and action and previous state and action.

6.5.2 Q Table

- The agent will use a Q-table to take the best possible action based on the expected reward for each state in the environment. In simple words, a Q-table is a data structure of sets of actions and states, and we use the Q-learning algorithm to update the values in the table.
- Fig. 6.5.1 shows Q-table.

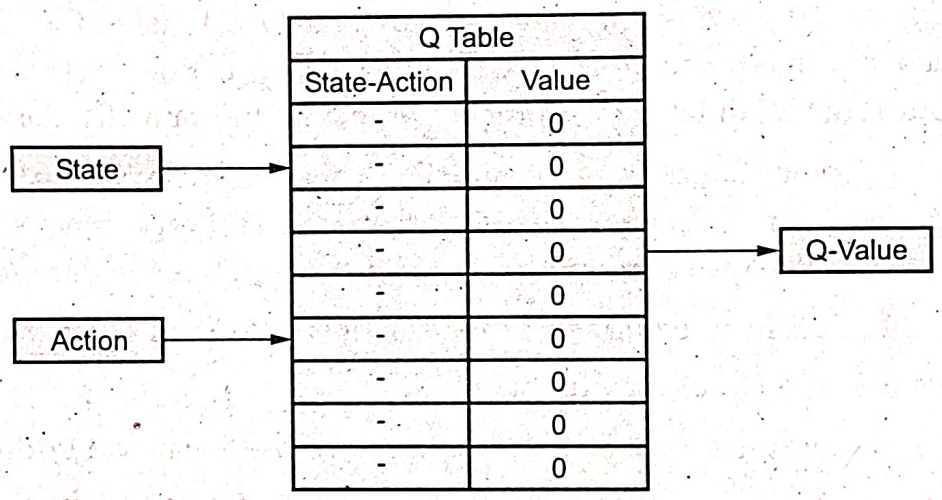


Fig. 6.5.1 Q-table

- This Q-table has a row for each state and a column for each action. Each cell contains the estimated Q-value for the corresponding state-action pair.
- We start by initializing all the Q-values to zero. As the agent interacts with the environment and gets feedback, the algorithm iteratively improves these Q-values until they converge to the Optimal Q-values. It updates them using the Bellman equation.

6.5.3 Q Functions

- The Q-function uses the Bellman equation and takes state(s) and action(a) as input. The equation simplifies the state values and state-action value calculation.

$$Q^{\pi}(s_t, a_t) = E\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid s_t, a_t\right]$$

Q-values for the state given a particular state

Expected discounted cumulative reward

Given the state and action

- Using the above function, we get the values of Q for the cells in the table. When we start, all the values in the Q-table are zeros.

6.5.4 Q Learning Algorithm

- In the Q-learning algorithm, the goal is to learn iteratively the optimal Q-value function using the Bellman optimality equation. To do so, we store all the Q-values in a table that we will update at each time step using the Q-learning iteration.

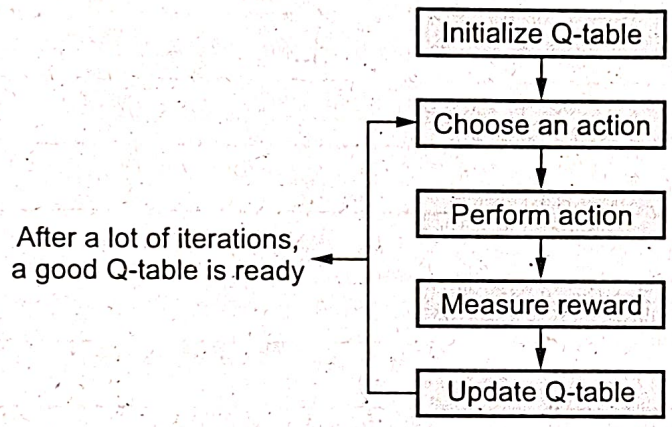


Fig. 6.5.2

- **Step 1 :** Initialize the Q-table : We will first build a Q-table. There are n columns, where n = number of actions. There are m rows, where m = number of states. We will initialise the values at 0.
- **Steps 2 and 3 :** Choose and perform an action
- This combination of steps is done for an undefined amount of time. This means that this step runs until the time we stop the training or the training loop stops as defined in the code.
- We will choose an action (a) in the state (s) based on the Q-Table. But, as mentioned earlier, when the episode initially starts, every Q-value is 0.
- **Steps 4 and 5 :** Evaluate
- Now we have taken an action and observed an outcome and reward. We need to update the function Q(s, a).

❑❑❑